

XR872 开发套件开发者手册

目录

版本说明.....	3
1 SDK 介绍.....	4
2 开发环境搭建.....	4
2.1 Windows Cygwin+Toolchain 安装与配置	4
2.1.1 Windows Cygwin 开发环境安装	4
2.1.2 Windows Toolchain 安装与配置	5
2.2 Linux Toolchain 安装与配置	5
3 SDK 编译.....	6
3.1 SDK 目录结构.....	6
3.2 SDK 关键 Makefile 和配置文件	7
3.3 SDK 编译命令及说明	8
3.4 SDK 编译	8
4 固件烧录	10
4.1 使用 XR872 SDK 自带 phoenixMC 烧录工具烧录	10
4.2 使用 iflymod 串口工具烧录.....	11
5 JYD SDK 接口及示例工程.....	14
5.1 JYD SDK 接口及说明.....	14
5.2 JYD SDK 软件开发使用指南	15
5.2.1 liboal	15
5.2.2 libjydconfig	15
5.2.3 libjydcloud.....	16
5.2.4 libjydaudio.....	16
5.3 jyd_evm 示例工程及开发说明	16
5.3.1 jyd_evm 示例工程介绍.....	16
5.3.2 jyd_evm 工程代码结构.....	16
5.3.3 jyd_sdk 功能模块说明	21
5.3.4 运行 jyd_evm 示例工程	27
5.3.5 jyd_evm 二次开发	28

5.3.6 jyd_evm 控制命令	30
5.3.7 提示音制作	31
5.4 串口通信及说明	32
6 工程配置	33
6.1 本地配置选项	33
6.2 工程 Makefile	34
6.3 链接脚本	35
6.4 镜像配置文件	35
6.5 创建新工程	36
6.5.1 创建工程目录	36
6.5.2 创建工程板级配置文件	36
6.5.3 创建工程链接脚本	37
6.5.4 创建工程镜像配置文件	38
6.5.5 修改工程 localconfig.mk	38
6.5.6 修改工程 Makefile	38
6.5.7 修改工程 prj_config.h	39
6.5.8 修改工程 command.c	40
7 常见问题及解决方法	40
7.1 如何替换 clientid	40
7.2 如何自定义唤醒词和离线命令词	43
7.3 如何开发自定义技能	43
7.4 jyd_evm 示例项目在哪里接收自定义技能控制指令	47
7.5 jyd_evm 示例项目如何处理离线命令	48

版本说明

版本	更改内容	时间
V1.0	初稿	2020/2/10
V1.1	修改	2020/4/23

1 SDK 介绍

XR872 开发套件 SDK 基于 XR872 官方 SDK 打造,可实现语音唤醒、iFLYOS 语音交互和串口数据交互等功能应用开发。本开发手册从开发环境搭建、SDK 编译、固件烧录、SDK 接口及调用方法及示例工程分析等方面详细介绍使用方法。

2 开发环境搭建

SDK 支持以下两种开发环境:

- Windows 环境: Cygwin + GCC
- Linux 环境: Ubuntu14.2 以上 + GCC

2.1 Windows Cygwin+Toolchain 安装与配置

2.1.1 Windows Cygwin 开发环境安装

方式一: 使用推荐的 Cygwin 开发环境 cygwin64.rar

为简化开发环境安装过程,本文推荐开发者使用 XF702 SDK 开发包中的 Cygwin 开发环境,安装步骤如下:

1. 解压 cygwin64.rar 到固定目录,目录路径不能包含中文字符或者空格;
2. 将 Cygwin.bat 添加到桌面快捷方式,双击桌面快捷方式图标即可进入 Cygwin 环境。

以上步骤完成 Cygwin 环境安装。

方案二: 网上下载安装

开发者也可通过下载安装 Cygwin 安装包进行安装,下载安装步骤如下:

1. 下载安装 [Cygwin](#), 双击运行“setup-x86_64.exe”或者“setup-x86.exe”,安装路径不含中文。

选择需要安装的工具包,必须选择安装以下工具:

binutils

gcc-core

gcc-java

make

cmake

git

xz

2. 安装完成后会在桌面产生“Cygwin”图标,双击图标即可进入 Cygwin 环境。

2.1.2 Windows Toolchain 安装与配置

SDK 开发包中包含 Toolchain 工具包 gcc-arm-none-eabi.rar, 为简化 Toolchain 安装和配置, 本文推荐开发者使用 XF872 SDK 开发包中提供的 Toolchain 工具包, 安装和配置步骤如下:

1. 将 XF702 SDK 源码及示例工程解压到已安装 cygwin 开发环境路径下的 home\Administrator (若无则新建) 目录下, 完整路径如示例所示:

```
cygwin64\home\Administrator\tools\gcc-arm-none-eabi-4_9-2015q2
--jyd_evm                      --示例工程
--jyd_evm_uart                 --示例工程, 自定义技能实现、串口通信、离线命令处理
--jyd_sdk                     --jyd_sdk
--jyd_sdk_sample               --jyd_sdk demo
--tools                        --交叉编译环境
--xradio-skylark-sdk-xradio-skylark-sdk-1.0.2 --XR872 SDK
```

2. Toolchain 编译工具配置

打开 xradio-skylark-sdk-xradio-skylark-sdk-1.0.2 目录 gcc.mk 文件, 第 8 行:

```
CC_DIR := ~/tools/gcc-arm-none-eabi-4_9-2015q2/bin
```

2.2 Linux Toolchain 安装与配置

1. 下载 Linux 版本的 Toolchain 压缩包
2. 进入控制台终端, 将 Toolchain 压缩包解压

```
# 切换到 Toolchain 压缩包所在目录
$ cd ~/tools
# 解压
$ tar -jxf gcc-arm-none-eabi-4_9-2015q2-20150609-linux.tar.bz2
```

3. 解压 XF702 SDK 源码文件 xradio-skylark-sdk-xradio-skylark-sdk-1.0.2.rar 到 xradio-skylark-sdk-xradio-skylark-sdk-1.0.2

4. SDK 关联 Toolchain 编译工具

打开 SDK 源码目录下 gcc.mk 文件, 第 8 行:

CC_DIR 配置路径应为:

```
CC_DIR := ~/tools/gcc-arm-none-eabi-4_9-2015q2/bin
```

以上操作完成 Linux 环境下 Toolchain 的安装和配置, 且 Toolchain 安装目录与“[sdk]/gcc.mk”中的“CC_DIR”变量一致 ([sdk]表示 SDK 根目录)。

3 SDK 编译

3.1 SDK 目录结构

```

.
├── bin                # bin 文件目录，存放预置 bin 文件，如 boot.bin 等
├── chip.mk
├── config.mk
├── gcc.mk
├── include            # 头文件目录，存放模块对外头文件
├── lib                # 库文件目录，存放预置库文件和编译生成的库文件
│   ├── libaac.a
│   ├── libairkiss_aes.a
│   └── .....
├── project            # 工程总目录
│   ├── bootloader    # bootloader 工程
│   ├── common        # 工程公用代码
│   ├── demo          # 演示工程总目录，该目录下每个目录对应一个工程
│   │   ├── hello_demo
│   │   ├── wlan_demo
│   │   └── .....
│   ├── example        # 示例工程总目录，该目录下每个目录对应一个工程
│   │   ├── uart
│   │   ├── wlan
│   │   └── .....
│   ├── image_cfg
│   │   └── image.cfg  # 默认镜像配置文件
│   ├── linker_script
│   │   └── gcc
│   │       ├── appos.ld  # 工程默认链接脚本
│   │       └── bootloader.ld  # bootloader 链接脚本
│   └── project.mk
├── test              # 测试工程总目录，该目录下每个目录对应一个工程
│   └── .....
├── src
│   ├── driver
│   │   ├── chip        # 芯片外设驱动
│   │   └── component   # 扩展外设驱动
│   ├── image          # image 模块
│   ├── kernel          # 内核
│   ├── ota             # OTA 模块
│   └── .....

```

```
├── .....
└── tools                # 镜像打包及烧录工具
```

3.2 SDK 关键 Makefile 和配置文件

SDK 的代码编译和镜像创建采用 Makefile 进行管理，关键 Makefile 和配置文件的说明见表 3-1。除特殊说明外，下文所有路径描述均为相对于 SDK 根目录的相对路径。

文件路径	功能描述
gcc.mk	(1) 定义通用编译规则，例如交叉编译工具链、编译选项、链接选项等。 (2) 一般情况下，用户只需要修改交叉编译工具链的路径定义“CC_DIR”。
config.mk	(1) 定义全局配置选项。所有全局配置选项均有默认值，其作用范围是 SDK 所有代码。 (2) 一般情况下，用户不需要修改此文件。如果用户需要改变全局配置选项的值，“project/[prj]/gcc/localconfig.mk”中可以进行重定义，从而覆盖默认值。
chip.mk	定义芯片配置选项，不允许用户修改。
src/Makefile	定义 SDK 模块，每个模块编译后均生成对应的库文件。
src/lib.mk	定义模块通用编译规则。
src/[module]/Makefile	(1) 模块编译的 Makefile，用于指定模块的库文件名、源文件等。 (2) [module]对应具体的模块路径，例如 image 模块的 Makefile 路径是：“src/image/Makefile”。
project/project.mk	定义工程的通用编译规则，一般无需修改。
project/[prj]/gcc/localconfig.mk	(1) 定义特定工程的本地配置选项。在此文件中定义的配置选项将会覆盖“config.mk”中定义的默认值。 (2) [prj]对应具体的工程路径，例如“hello_demo”工程对应的路径为“project/demo/hello_demo/”
project/[prj]/gcc/Makefile	特定工程的 Makefile，用于指定工程板级配置、源文件、链接脚本、镜像、配置文件等。
project/common/prj_conf_opt.h	(1) 定义工程功能选项的默认配置（即针对所有工程的默认配置）。每个功能选项对应一个宏定义，例如中断服务程序栈大小 (PRJCONF_MSP_STACK_SIZE)、主线程栈大小 (PRJCONF_MAIN_THREAD_STACK_SIZE)等。 (2) 该文件中定义的宏，“project/”目录下的所有源文件均可见。

	(3) 一般情况下，用户不需要修改此文件。如果特定工程需要改变某功能选项的值，可在“project/[prj]/prj_config.h”中进行重定义，从而覆盖默认值。
project/[prj]/prj_config.h	(1) 定义特定工程的功能选项配置值。在此文件中定义的功能选项值将会覆盖“project/common/prj_conf_opt.h”中定义的默认值。 (2) [prj]对应具体的工程路径，例如“hello_demo”工程对应的路径为“project/demo/hello_demo/”。 (3) 每个工程都必须实现本工程的“prj_config.h”文件。 (4) 该文件中定义的宏，“project/”目录下的所有源文件均可见。

表 3-1 SDK 关键 Makefile 和配置文件

3.3 SDK 编译命令及说明

代码编译命令和命令的说明见表 3-2：

命令	命令说明
make config	(1) 执行 SDK 基础配置（根据提示，进行芯片型号等配置选择）该配置对所有工程生效。如果需要更改 SDK 基础配置，可再次执行此命令。 (2) 正确执行该命令后，将在 SDK 根目录下生成“.config”文件。 (3) 必须执行该命令后才可以执行其他编译命令。
make config_clean	删除“make config”命令生成的文件“.config”。
make lib	编译“src”目录下的模块，生成库文件并复制到“lib”目录。
make lib_clean	删除“make lib”命令在“src”目录下生成的中间编译文件。
make lib_install_clean	删除“make lib”命令在“lib”目录下生成的库文件。
make	编译工程代码，生成可执行文件（位于“project/[prj]/gcc/”目录）。
make clean	删除“make”命令生成的所有文件。
make image	创建镜像文件（位于“project/image/[chip]/”目录，[chip]对应为“make config”命令执行时选择的芯片型号，例如“xr872”、“xr808”）。
make image_clean	删除“make image”命令生成的所有文件。
make build	相当于执行命令“make lib && make && make image”。
make build_clean	相当于执行命令“make image_clean clean lib_clean lib_install_clean”。
make objdump	生成反汇编文件，用于代码分析和调试。
make size	显示工程 ELF 文件的 size 信息，如 text 段、data 段、bss 段大等。

表 3-2 代码编译命令和命令说明

3.4 SDK 编译

代码编译前需要在“gcc.mk”中设置正确的 GCC 交叉编译工具链路径，例如：

```
CC_DIR := ~/tools/gcc-arm-none-eabi-4_9-2015q2/bin
```

所有代码编译和镜像创建命令均需在 Cygwin 环境或者 Linux 终端执行，以 jyd_evm 项目工程为例，编译步骤：

1. 切换到工程编译目录：

```
$ cd jyd_evm/gcc
```

2. 执行 SDK 基础配置

```
$ make config
*
* XRADIO SDK Configuration
*
Chip
  1. XR872
  2. XR808
choice[1-2]: 1

选择芯片型号为 XR872  1

External high speed crystal oscillator
  1. 24M
  2. 26M
  3. 40M
choice[1-3]: 3

选择 40M  3
```

3. 编译镜像文件

第一次编译或 **make clean** 之后需要执行操作：

运行“release_make”可执行文件产生版本号的 “verno.h” 并打包执行 **make** 和 **make image** 命令：

```
$ ./release_make
```

手动编译步骤：

首先编译“src”目录下的模块，生成库文件并复制到“lib”目录；

```
$ make lib
```

其次编译工程代码，生成可执行文件位于“project/[prj]/gcc/”目录；

```
$ make
```

最后创建镜像文件：

```
$ make image
```

生成可执行文件位于“project/image/[chip]/”目录，[chip]对应为“make config”命令执行时选择的芯片型号，例如“xr872”、“xr808”。

4 固件烧录

4.1 使用 XR872 SDK 自带 phoenixMC 烧录工具烧录

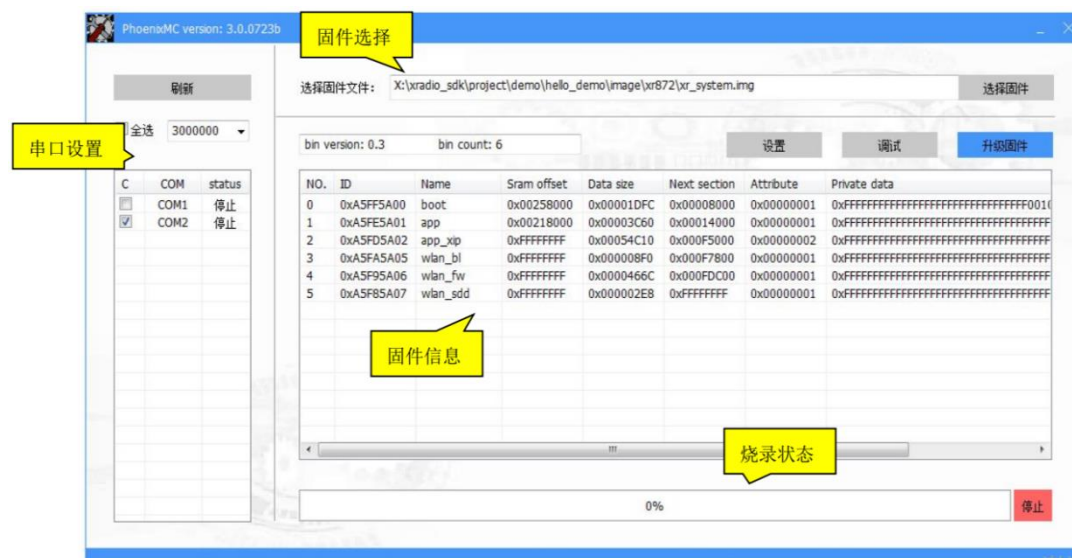


图 4-1

Windows 版本的烧录工具（GUI 版本）位于“SDK/tools/phoenixMC.exe”，Linux 版本的烧录工具（命令行版本，无 GUI 版本）位于“SDK/tools/phoenixMC”。具体步骤如下：

1. 串口连线

将 XF702 WIFI 语音模组开发板通过专用下载线连接电脑，开发板上电开机后处于升级模式，无需另外操作，PhoenixMC 烧录工具识别到串口设备为连接成功。

2. 串口设置

点击左上角的“刷新”按钮可刷新已连接串口设备列表，勾选开发板对应的 COM 口。为了避免烧录速度过慢，建议波特率选择大于等于 921600。

3. 固件选择

点击“选择固件”按钮选择需要烧录的固件文件（扩展名为“.img”），固件信息栏会显示出当前固件的详细信息。另外，通过拖拽方式将固件直接拖入工具界面也可以达到同样的效果。

4. 启动烧录

点击“升级固件”按钮启动固件烧录。烧录状态栏显示当前选定串口对应设备的烧录进度和状态。当烧录成功时，进度条会达到 100%的进度并显示为绿色；

当烧录失败时，进度条显示为红色并报告错误。

4.2 使用 iflymod 串口工具烧录

1. 安装 CP210x_Windows_Drivers 串口驱动程序

解压 CP210x_Windows_Drivers 串口驱动包，根据操作系统选择对应的版本安装，安装过程如下图所示：



图 4-2

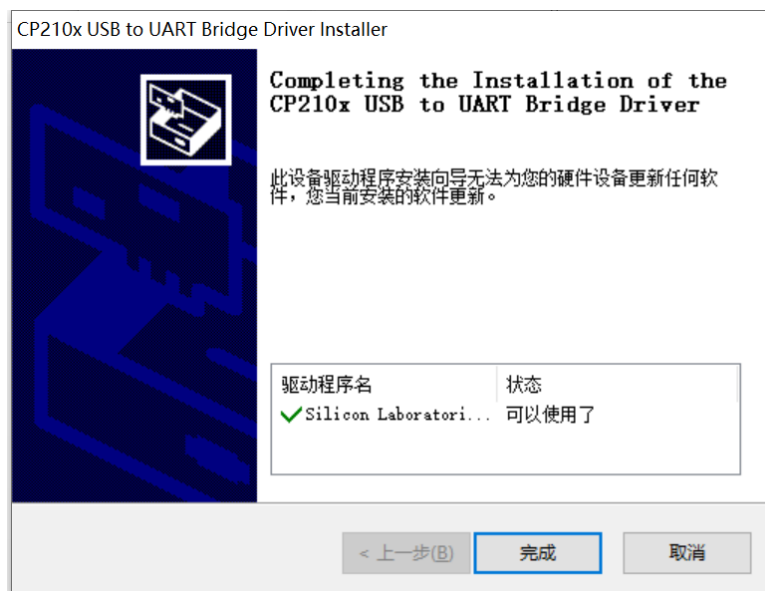


图 4-3

2. 安装 iflymod 串口工具

运行 iuarttoolsSetup6.9.exe 安装 iflymod 串口工具，按安装步骤完成安装：

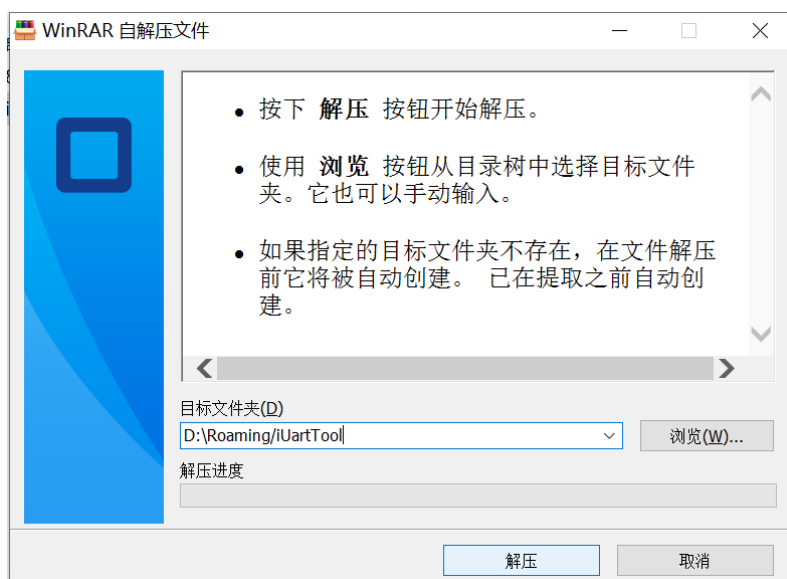


图 4-4

安装成功后再电脑桌面生成 iflymod 串口工具图标。

3. iflymod 串口工具连接设备

- 双击 iflymod 串口工具图标打开串口工具
- 将串口线连接电脑 usb 接口

在串口下拉列表中有两个串口，离线模块串口和在线模块串口，配网操作需要选择打开在线模块串口并发送配网命令。

如何区分在线模块和离线模块串口？

串口标识：UART Bridge : Standard

4. 设备配网

串口送配网指令

通过 iflymod 串口工具发送配网指令到设备端实现配网。

配网指令：

```
jyd connect TP-LINK_8DCB 1234567890
```

其中 TP-LINK_8DCB 为 WIFI 名称，1234567890 为 WIFI 密码，根据实际的 WiFi 名称和热点设置，如下图所示：

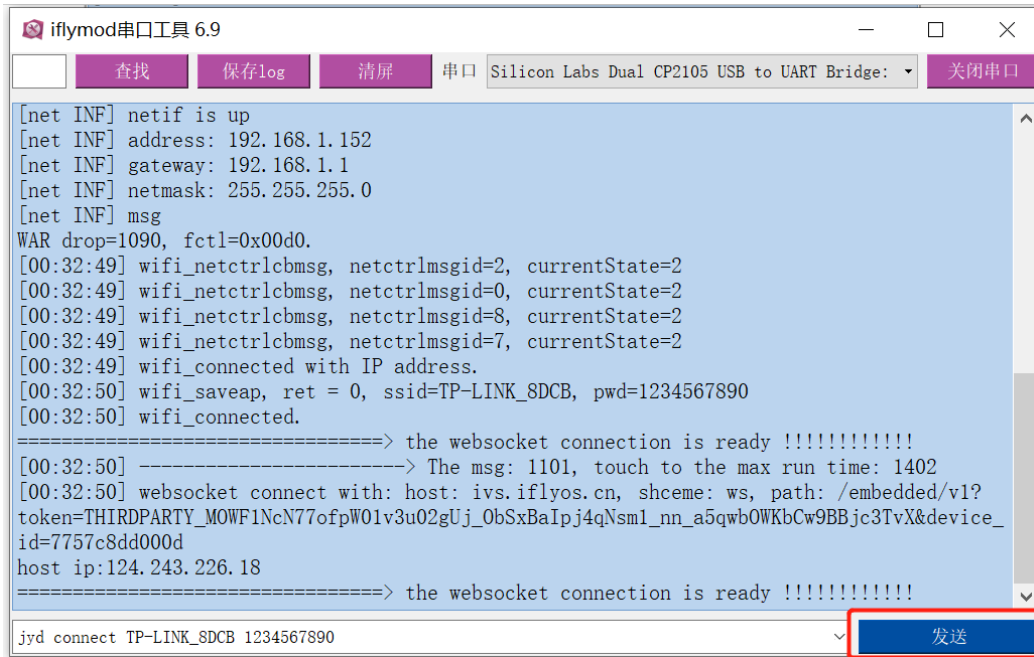


图 4-5

网络正在连接语音提醒：网络连接中，请稍等

网络连接成功语音提醒：网络连接成功

配网成功后设备发出配网成功语音提示，配网操作完成。

5. 烧录软件

烧录命令：

```
jyd ota http http://192.168.1.133:8000/upload/xr_system.img
```

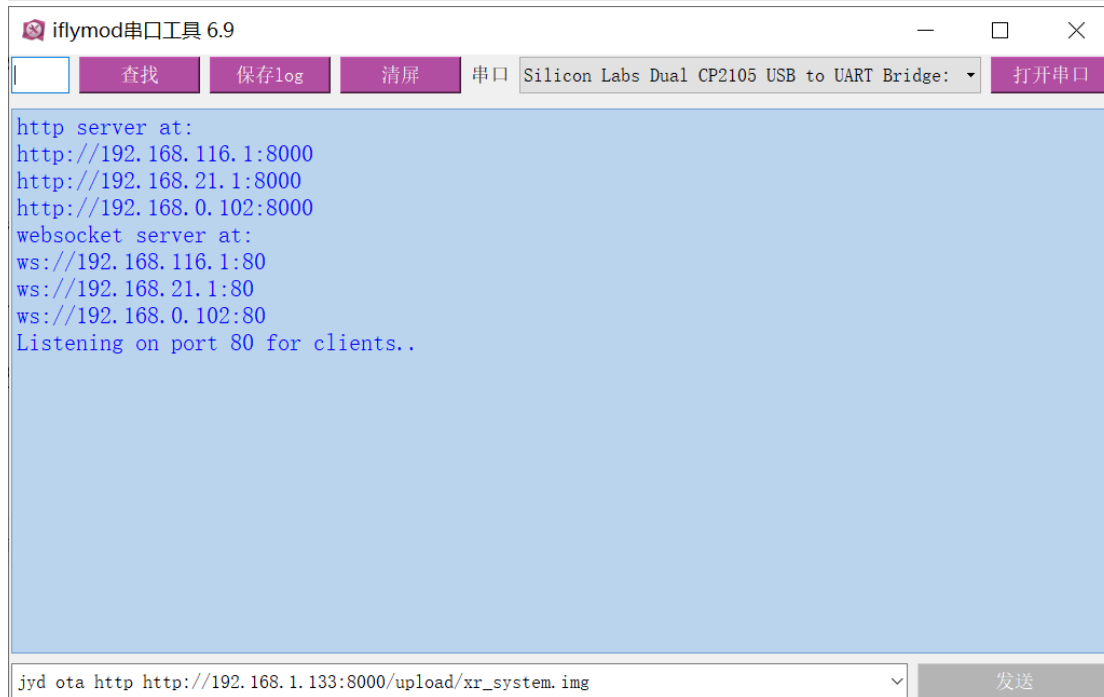
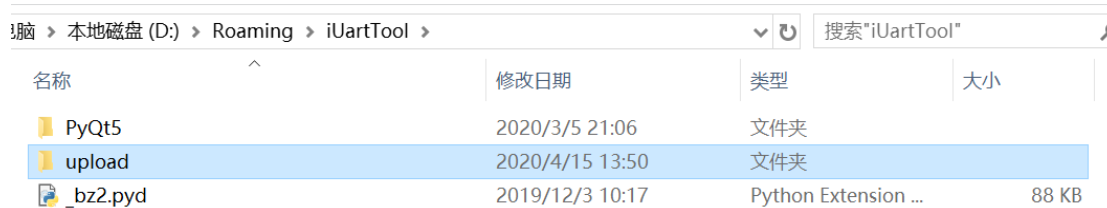


图 4-6

命令说明：

1. http://192.168.1.133 电脑 ip 地址，端口默认未 8000。

2. upload 目录为 iflymod 串口工具安装目录下用了存放固件的文件夹，若无则创建，如：D:\Roaming\iUartTool\upload。



本地磁盘 (D:) > Roaming > iUartTool >			
名称	修改日期	类型	大小
PyQt5	2020/3/5 21:06	文件夹	
upload	2020/4/15 13:50	文件夹	
_bz2.pyd	2019/12/3 10:17	Python Extension ...	88 KB

图 4-7

3. xr_system.img 为需要烧录固件名称。

5 JYD SDK 接口及示例工程

5.1 JYD SDK 接口及说明

金源德 jyd_sdk 是基于 xr872 官方 SDK，为结合科大讯飞 iFlyOS，实现并提供离线唤醒，录音，云端交互等等系列人工智能的解决方案的更具有针对性、更高级别的 SDK。

JYD SDK 的架构概括如图-5-1 示：



图 5-1

5.2 JYD SDK 软件开发使用指南

JYD SDK 基于 xradio-skylark-sdk-xradio-skylark-sdk-1.0.2（或更新的版本）打造，其代码结构为：

```
jyd_sdk
├─libjydaudio
├─libjydcloud
├─libjydconfig
├─liboal
└─jydsdk_common.h
```

其中依赖关系为：

```
App -> libjydaudio -> libjydcloud -> libjydconfig -> liboal
```

5.2.1 liboal

liboal 为基于 C 语言的操作系统抽象层，便于兼容各个操作系统(windows, Linux, RTOS)之间的调用带来的接口差异。liboal 的主要提供如下系统级底层服务接口。

- 系统 time, timer 调度抽象。
- 网络 socket 接口抽象。
- 标准 c 库引用。
- 字符串操作，URL 解析等。
- 内存管理操作（目前只管理 psram）。
- 声卡驱动接口抽象。
- 线程，信号量，互斥量等抽象。
- 线性内存块，互锁内存块操作。
- 系统队列。
- 带优先级的消息队列（不常用）。
- 消息映射表（消息泵）。

liboal 中值得专门介绍的是消息泵的使用，具体参考 oal.h 带来的接口说明。

5.2.2 libjydconfig

libjydconfig 提供类似于 COM 组件的 WiFi 管理功能，以及 Flash 信息读写功能。

通过 iflymod_wifi.h 提供 WiFi 管理功能，目前包括了：

- WiFi 扫描
- WiFi 连接中
- WiFi 已连接，断开

- WiFi 微信配网
- WiFi 声波配网，开放的接口，支持添加其他各种配网方式的组件

通过 `nvrmmgr.h` 提供 Flash 信息读写接口主要包括存储读写：

- 各种授权码信息：device id，token，唤醒库授权码等。
- 配网过的 WiFi 热点名称，密码等。
- 各种系统信息掉电记忆：音量等。

5.2.3 libjydccloud

libjydccloud 提供云端后台的能力，目前 libjydccloud 提供 iFLYOS 的 EVS，IVS 的部分已实现的接口，包括：

- 授权接口
- 登录登出接口
- 交互请求接口
- 反馈接口
- 杂类（例如获取闹钟个数，强制对时等）接口

详细请参考 `iflymod_iflyos.h` `iflymod_iflyosIVS.h` 的调用。

5.2.4 libjydaudio

libjydaudio 提供接口：

- 录音、唤醒、识别传输
- 音频播放、提示音播放的功能

详情请参考 `iflymod_recorder.h` `iflymod_playlist.h`

5.3 jyd_evm 示例工程及开发说明

5.3.1 jyd_evm 示例工程介绍

jyd_evm 工程是基于 XR872 SDK 开发并集成科大讯飞 iFLYOS 智能语音交互系统的应用软件项目，实现了基本的语音唤醒、交互，OTA 的功能，同时支持常用的控制台命令操作，开发者可用于系统调试。

5.3.2 jyd_evm 工程代码结构

```
|board
|└─board.*          #工程的板级配置代码，硬件管脚功能配置
```

```

├─gcc
│   ├──localconfig.mk    # 工程的配置选项，可覆盖全局默认配置
│   └─Makefile          # 工程的 Makefile，可指定 src、board config、ld、image cfg
├─image
│   └─xr872
│       ├──tone.h        # 工程的提示音定义文件
│       └─tone.bin       # 工程的提示音镜像
├─libwakeup
│   └─*..*               # 唤醒模块的库和头文件
├─command.c              # 工程的控制台命令入口和命令定义
├─command.h
├─main.c                  # 工程的入口，完成平台初始化调用流程。
├─aip*..*                 # 数码管显示屏显示驱动代码
├─btn.*                   # 按键处理模块
├─iclock.c                # 时间显示模块
├─iflymod_command.c      # jyd_sdk 特有命令行命令代码的实现
├─main_task.c             # 主工作线程模块，对 jyd_sdk 的调用方法都在这里
├─prj_config.h            # 工程的功能选项配置，主要用于功能的选择, PRJCONF_NET_EN:
│                           可选项，配置是否开启网络功能，注意本工程必须开启此项。
└─readme.md              # 工程的说明文档

```

程序入口

```

int main(void)
{
    // 平台初始化
    platform_init();

    // 初始化公共数据(全局数据)
    jyd_sdk_common_data_init();

    // 主任务启动(消息泵)
    main_task();

    iClock_Init();
    return 0;
}

```

主工作任务

```

void main_task()
{
    // 播放列表初始化，播放欢迎词
}

```

```
iflymod_playlist_init();
iflymod_playlist_add_promp_tone(10);
//启动消息泵（消息映射表）
message_map_handle = OAL_MsgMapStart( main_task_message_map_functions,
MAIN_LOOP_TIME_OUT,50, OAL_THREAD_PRIORITY_LOW_HIGH);
//启动WIFI管理模块
iflymod_wifi_start();
//启动录音唤醒模块
iflymod_audio_recorder_start();

// WiFi模块消息依附
iflymod_wifi_attach_msg(MSG_WIFI_STATE_REPORT, wifi_callback);

// 云端模块消息依附
iflymod_cloud_attach_msg(IFLYOS_RESPONSE_TYPE_LOGIN, cloud_connect_callback);
iflymod_cloud_attach_msg(IFLYOS_RESPONSE_TYPE_LOGOUT, cloud_connect_callback);
iflymod_cloud_attach_msg(IFLYOS_RESPONSE_TYPE_CHANGE_VOLUME, volume_callback);
iflymod_cloud_attach_msg(IFLYOS_RESPONSE_TYPE_ERROR, error_callback);
iflymod_cloud_attach_msg(IFLYOS_RESPONSE_TYPE_CUSTOM, custom_callback);
iflymod_cloud_attach_msg(IFLYOS_RESPONSE_TYPE_PLAY_URL, http_url_callback);

// 音频模块消息依附
iflymod_audio_recorder_attach_msg(MSG_AUDIO_RECOGNIZE_STARTED, recorder_callback);
iflymod_audio_recorder_attach_msg(MSG_AUDIO_RECOGNIZE_STOPPED, recorder_callback);
iflymod_audio_recorder_attach_msg(MSG_AUDIO_RECOGNIZE_VOICE_SEND_FAILED,
recorder_callback);
iflymod_audio_recorder_attach_msg(MSG_AUDIO_OFFLINE_COMMAND, recorder_callback);

// 按键事件回调
btn_register_callback(jyd_sdk_common_data.button_gpio, button_callback);
}
```

消息映射表（消息泵）

jyd_sdk 主要采用消息驱动的方式来完成所有的用户级应用和逻辑，类似于 windows 的 MFC 的消息映射结构，采用“发送”---“响应”的结构。消息泵在创建的时候会拥有一个独立的工作线程。凡是事务级的模块间通讯均可采用此软件框架，有以下优点：

- 可以将繁琐庞大的用户逻辑清晰处理开来，省去在 RTOS 这种系统上做很多线程级的底层处理和事件处理。
- 可以将复杂的模块耦合降到最低，保证模块的独立化，清晰化。消息泵的主要接口：（参考 liboal 中的 oal_msgmap.h）

示例代码：

```
_this.message_map_handle=OAL_MsgMapStart(main_task_message_map_functions,
MAIN_LOOP_TIME_OUT, 50, OAL_THREAD_PRIORITY_LOW_HIGH);
```

消息定义:

```
enum {
    MSG_MAIN_TASK = (OAL_MSG_MAP_MESSAGE_USER_BASE + 1000),
    MSG_MAIN_TASK_WIFI_DISSCONNECTED,
    MSG_MAIN_TASK_WIFI_CONNECTED,
    MSG_MAIN_TASK_BUTTON_EVENT,
    MSG_MAIN_TASK_UPGRADE_FIRMWARE,
    MSG_AMIN_TASK_TASK_POWEROFF,
    MSG_MAIN_TASK_BUTTON_EVENT_EX,

    MSG_MAIN_TASK_MAX, // to be the end message id.
};
```

消息响应函数:

```
const OAL_CALL_FUNCTION_LIST main_task_message_map_functions[] = {
    {OAL_MSG_MAP_MESSAGE_BEGIN_HOOK, OnMsgBeginHook, 0},
    {OAL_MSG_MAP_MESSAGE_AFTER_HOOK, OnMsgAfterHook, 0},
    {OAL_MSG_MAP_MESSAGE_TIMEOUT_HOOK, OnMsgTimeoutHook, 0},
    ///////////////////////////////////////////////////////////////////
    {MSG_MAIN_TASK_WIFI_DISCONNECTED, OnMsgWifiDisconnected, 0},
    {MSG_MAIN_TASK_WIFI_CONNECTED, OnMsgWifiConnected, 0},
    {MSG_MAIN_TASK_BUTTON_EVENT, OnMsgButtonEvent, 0},
    {MSG_MAIN_TASK_UPGRADE_FIRMWARE, OnMsgUpgradeFirmware, 0},
    {MSG_MAIN_TASK_POWEROFF, OnMsgPoweroff, 0},

    {0,0,0}
}
```

注意的一些概念:

- 同一个消息泵里面的每个消息是串行执行的，先发的消息先被执行。
- 示例里面参数 50 代表消息泵队列里面最多缓存 50 条消息，可根据业务的复杂度调整该参数，若缓存的消息数量超过 50，不管是 Post 方法还是 Send 方法均会被阻塞，直到最顶端的消息被处理完毕。所以要求应用尽量花短的时间处理消息。
- 三个钩子函数:

```
{OAL_MSG_MAP_MESSAGE_BEGIN_HOOK, OnMsgBeginHook, 0},
{OAL_MSG_MAP_MESSAGE_AFTER_HOOK, OnMsgAfterHook, 0},
{OAL_MSG_MAP_MESSAGE_TIMEOUT_HOOK, OnMsgTimeoutHook, 0},
```

- 任意一个消息到来时，begin hook 会首先被调用；任意一个消息结束时，after hook 会被调用；无任何消息时，timeout hook 会被调用，其中超时

时间在启动函数里面传参定义。若启动后想加快消息泵的闲置触发速度，可用 `set timeout` 接口重新指定。

示例代码中，利用 `begin hook`, `after hook` 来统计某个消息执行的时间。

消息发送

```
OAL_RESULT OAL_MsgMapSendMsg(OAL_HANDLE handle, int id, int wParam, int lParam, void *data,
OAL_RESULT OAL_MsgMapPostMsg(OAL_HANDLE handle, int id, int wParam, int lParam, void *data,
```

- 消息的发送可以是异步的，也可以是同步的，其中 `Send` 表示接受方处理完该消息才返回，`Post` 表示将消息发给对方，不管对方是否执行了，立即返回。
- 消息泵同步发送消息不可使用 `Send` 方法，否则会造成死锁，`Send` 方法只能异步调用。
- 值得提出的是，用 `Post` 和 `Send` 方法发送消息时，假如 `data_len` 的参数不为 0，则消息泵会自动申请 `data_len` 长度的内存进行传递。并在消息执行完后自动回收。假如我们传递的内容 `data` 是个全局地址或者是常数地址，则 `data_len` 可以指定为 0，接收者会访问到该区域内存，且不会丢失，避免系统开销。

消息依附

当某处程序希望响应其他消息泵的消息时，则使用消息依附：

```
OAL_RESULT OAL_MsgMapAttach(OAL_HANDLE handle, int id, OAL_CALL_FUNCTION pfunc);
```

示例代码中，`main_task` 启动时，依附消息 `IFLYOS_RESPONSE_TYPE_LOGIN`，表示云端登录成功后，在消息响应函数：`cloud_connect_callback` 中播放“云端登录成功”的提示音。

又例如依附 `MSG_WIFI_STATE_REPORT` 消息，来获得 WiFi 管理模块网络连接成功的事件通知。

消息委托

- 消息委托和消息依附的工作原理和接口是一致的，区别在于消息委托的消息不在被委托的消息泵中响应，但会被执行，执行的前提是有人委托了该消息。应用场景是被委托的消息泵并不关心该消息，实现了软件模块的完全独立拆分，但运行机制又是紧密合作的，在消息委托中，需要做几个步骤：
- 在外部串口通信的示例中，串口收到底层硬件中断，处理并接收到串口信息，这个时候，需要回复对方一些信息，但又不能在接收中断处理函数中发送给对方，假如重新开个线程的话对系统资源消耗和内存消耗都很大，麻烦且不省心，这个时候则需要委托 `main_task` 来处理：

定义一个与被委托消息，并保证该消息不与 `main_task` 的消息定义重复：

```
Enum {
```

```

MAIN_TASK_DELEGATE_MSG_SEND = MSG_MAIN_TASK_MAX+1,
MAIN_TASK_DELEGATE_MSG_RECV,
};

```

定义处理函数：

```

static void _extcom_send (int id, int wParam, int lParam, void *data, int data_len)
{
    extcom_sendbytesEX(data, data_len);
    OAL_log("OnMsgExtuartcomSendDat, data_len=%d\r\n", data_len);
}

```

委托 main_task 处理该消息：

```

Main_task_attach_msg(MAIN_TASK_DELEGATE_MSG_SEND, _extcom_send);

```

最后，在接收中断处理里面发送该消息即可，main_task_post_message。

5.3.3 jyd_sdk 功能模块说明

1 libjydaudio（音频库）

录音和唤醒模块

iflymod_recorder.h

```

enum{
    MSG_AUDIO_ = (OAL_MSG_MAP_MESSAGE_USER_BASE+200),
    MSG_AUDIO_RECOGNIZE_STARTED,
    MSG_AUDIO_RECOGNIZE_STOPPED,
    MSG_AUDIO_RECOGNIZE_VOICE_SEND_FAILED,
    MSG_AUDIO_OFFLINE_COMMAND,
};
OAL_RESULT iflymod_audio_recorder_start();
OAL_RESULT iflymod_audio_recorder_stop();
OAL_BOOL iflymod_audio_recorder_is_recognizing();
OAL_RESULT iflymod_audio_recorder_cancel_current_recognize();
OAL_RESULT iflymod_audio_recorder_attach_msg(int id, OAL_CALL_FUNCTION pfunc);

```

以上接口提供：

- 启动录音唤醒
- 停止录音唤醒
- 消息依附接口和消息定义

在我们设计 JYD SDK 之初，考虑到硬件搭配的多样性，录音功能和唤醒功能的实现被定义成了两组弱函数，方便进行重载。我们利用 C99 的特性，将这

两个组件实现为虚组件，但并非纯虚组件，也就是说我们在 JYD SDK 内部已经对这两个组件有了默认实现：

1. “唤醒器”在外部应用程序不实现的情况下，只支持手动唤醒。
2. “录音机”在外部应用程序不实现的情况下，支持 liboal 的声卡抽象实现，目前是采用 xr872 的 internal codec 进行录音，为了避免过多的参数带来的复杂性，我们一致规定，“录音机”统一采用 16000 采样率，16 位每采样的参数进行，只有声道数一个参数是可变的。

“唤醒器”特点：

- 接口可被完全替换。
- 任意一种实现，必须实现手动唤醒接口。

目前唤醒支持软唤醒和硬唤醒，并同时支持手动唤醒。唤醒模块在 libwakeup 中实现，该系列弱函数可实现软硬唤醒的继承替换工作。

唤醒模块的弱函数定义：

```
_weak OAL_RESULT audio_wakeup_init()
>{
}

//return wakeup_type, // 0: none, 1:wakeup, >1: other keyword
__weak int audio_wakeup_process(char *pcm, unsigned int pcm_len, char *pcm_1ch, unsigned int *pcm_1ch_len)
>{
}

>__weak void audio_wakeup_manual_wakeup()
{
}

>__weak int audio_wakeup_ires_div_point()
>{
}
```

在有硬件唤醒的案例中，则重载这些函数即可实现。具体参考 libwakeup 的实现，其中注意 audio_wakeup_manual_wakeup 函数无论在任何情况调用会产生直接唤醒的效果，这样可以与语音的唤醒词并行存在。

另外，唤醒处理接口（audio_wakeup_process）可以返回唤醒的状态，并返回唤醒序号，同时满足真正的“唤醒”和“其他离线命令词”。

在我们的 libjydaudio 的消息 MSG_AUDIO_OFFLINE_COMMAND 的中依附即可获得。

“录音机”特点：

我们一致认为，“唤醒器”在一个项目中是唯一的，所以“唤醒器”是全局单例模式；考虑到与“唤醒器”的不同，录音机可以被调用为多实例模型，（例如多路麦克风，多声卡，含回声消除等等或者其他更复杂的情况）“录音机”相对唤醒器要简单的多，其弱函数定义如下：

```
__weak void* a_audio_recorder_open(unsigned int channels)
{
```

```

}
__weak void a_audio_recorder_record(void *p, char* pcm, unsigned int pcm_byte_len)
{
}
__weak void a_audio_recorder_close(void *p)
{
}

```

播放列表模块

iflymod_playlist.h

```

OAL_RESULT iflymod_playlist_init();
OAL_RESULT iflymod_playlist_uninit();
OAL_RESULT iflymod_playlist_attach_msg(int id, OAL_CALL_FUNCTION pfunc);
OAL_BOOL iflymod_playlist_is_playing();

OAL_RESULT iflymod_playlist_add(PPLAYLIST_ITEM *item);
OAL_RESULT iflymod_playlist_add_promp_tone(int index);
OAL_RESULT iflymod_playlist_add_only(PPLAYLIST_ITEM *item);
OAL_RESULT iflymod_playlist_resume();
OAL_RESULT iflymod_playlist_break();
OAL_RESULT iflymod_playlist_pause();
OAL_RESULT iflymod_playlist_stop();
int iflymod_playlist_waitings();

```

播放列表可以在本地维持一个音频播放的序列逻辑，包括播放提示音，播放网络资源等等要播放的所有音频内容。

播放列表规定了一个优先级，根据类型依次排序：

```

typedef enum{
    PLAY_LIST_ITEM_TYPE_NONE           = 0,
    PLAY_LIST_ITEM_TYPE_URL            = 10,
    PLAY_LIST_ITEM_TYPE_VOICE          = 20,
    PLAY_LIST_ITEM_TYPE_ALARM          = 30,
    PLAY_LIST_ITEM_TYPE_PROMP          = 40,
}PLAYLIST_ITEM_TYPE;

```

- 当要播放一个音频资源时，按照优先级进行取舍，我们规定提示音是优先级最高的，其次是闹钟播放，TTS 语音播放，网络歌曲播放。一共 4 种类型。我们规定提示音是存在设备 Flash 中的，是重要的提示声音，不可丢弃。假如正在播放提示音时，网络推送了一个音乐，则会排队等到提示音播放完毕，然后再自动播放后面排队的资源，举个例子，正在播放提示音时，先来了一个网络歌曲，然后又来了一个闹钟，则提示音播放完毕后剩下的播放顺序为：闹钟，歌曲。
- 当高优先级的内容来临时，会打断当前播放的内容，去播放高优先级的

内容。

- 当播放的类型为 URL 类型时，则会保持断点，等高优先级的内容播放完毕，再从断点处继续播放。
- 当非 URL 类型的内容被打断时，则会丢弃，以后不再播放。相同优先级的内容来时，会排队等待当前播放完毕。
- 当播放的 item 属性存在“can be ignore”属性时，则相同优先级的其他资源会被播放，并丢弃当前资源，即便存在该属性，低优先级的资源来临时，也不会“被忽略”或者打断。

应用场景举例：当正在扫描网络时，联网速度很快，同时启动“网络连接中...”和“网络连接成功”的提示音播放，则会丢弃“网络连接中...”的提示音，直接播放“网络连接成功”。

值得注意的接口说明：

- flymod_playlist_is_playing 假如列表中有等待或者正在播放，则返回 true。
- iflymod_playlist_add 向播放列表中添加一个内容，假如目前播放列表是闲暇状态，则立即播放当前添加的内容。
- iflymod_playlist_add_promp_tone 是对上面接口的封装，添加一个提示音，参数是序号。
- iflymod_playlist_add_only 是向播放列表添加内容，但不要立即播放。
- iflymod_playlist_break 是仅仅打断当前播放不向外汇报，其他播放控制接口均会向外汇报状态，例如告诉云端当前的资源播放的情况。

2 libjydccloud（云端库）

负责后台鉴权，登录，交互，语音上传，指令下达等。

```
typedef struct{
    char *url;
    char *resource_id,
    int is_tts_request_by_user;
    char *tts_url;
    char *tts_resource_id;
    char *music_url;
    char *music_resource_id;
    char *alarm_url;
    char *alarm_resource_id;
    int time_stamp;
    int volume;
    int error_code;
    char *custom_string;
    char *intermediate_text;
    char *reply_key;
    char *tts_text;
    int play_control_status;
}JYD_CLOUD_RESP_DATA;

OAL_RESULT iflymod_cloud_start();
```

```
OAL_RESULT iflymod_cloud_stop();
OAL_RESULT iflymod_cloud_attach_msg(int id, OAL_CALL_FUNCTION pfunc);
OAL_BOOL iflymod_is_loggedin();
void iflymod_wifi_enable_log(OAL_BOOL enable);

OAL_RESULT iflymod_cloud_send_request(int request_id, char *resource_id, char *to_tts_text);
OAL_RESULT iflymod_cloud_send_voice(char *pcm_640bytes, int nBytes);
OAL_RESULT iflymod_cloud_send_voice_end(OAL_BOOL is_end);
```

其中 iflymod_cloud_send_xxxxx 接口为主动发送请求接口。
所有返回的响应均做消息依附，云端返回的事件定义为：

```
// response type
enum{
    IFLYOS_RESPONSE_TYPE_NONE,
    IFLYOS_RESPONSE_TYPE_LOGIN,
    IFLYOS_RESPONSE_TYPE_LOGOUT,
    IFLYOS_RESPONSE_TYPE_VAD,
    IFLYOS_RESPONSE_TYPE_INTERMEDIATE_TEXT,
    IFLYOS_RESPONSE_TYPE_ERROR,
    IFLYOS_RESPONSE_TYPE_TIME_STAMP,
    IFLYOS_RESPONSE_TYPE_CHANGE_VOLUME,
    IFLYOS_RESPONSE_TYPE_PLAY_URL,
    IFLYOS_RESPONSE_TYPE_CUSTOM,
    IFLYOS_RESPONSE_TYPE_REPLY_KEY,
    IFLYOS_RESPONSE_TYPE_PLAY_CONTROL,
}
```

在依附的消息中，我们规定返回的内容均包含在 JYD_CLOUD_RESP_DATA 这样一个数据结构中，根据返回的 id 取相应数据即可。其中，音频模块已经 attach 了其相关事件，云端模块的接口较简单，可以参考头文件阅读。

3 libjydconfig（系统配置库）

关于 Flash 信息存储，参考 nvrammgr.h 的定义即可，纯静态的读写接口，读取和存储用户信息。

WiFi 连接模块参考 iflymod_wifi.h 的定义，WiFi 管理模块采用 COM 组件思路实现管理，每个组件代表一种状态机。由于接口简单，不再过多阐述。

```
enum{
    WIFI_STATE_REPRORT_ = 0,
    WIFI_STATE_REPRORT_CONNECTED,
    WIFI_STATE_REPRORT_DISCONNECTED,
    WIFI_STATE_REPRORT_CONNECTING,
    WIFI_STATE_REPRORT_CONFIG_FAILED,
    WIFI_STATE_REPRORT_CONNECT_FAILED,
```

```

WIFI_STATE_REPRORT_PASSWORD_FAILED,
WIFI_STATE_REPRORT_RECEIVED_SSID_PASSWORD,
}

enum{
    MSG_WIFI_BASE = (OAL_MSG_MAP_MESSAGE_BASE+400)

    MSG_WIFI_SYSTEM_CALLBACK,
    MSG_WIFI_USER_FORCE_CONTORL,
    MSG_WIFI_STATE_REPORT,
}

OAL_RESULT iflymod_wifi_start();
OAL_RESULT iflymod_wifi_post_msg(int id, int value1, int value2, void *data, int data_len);
OAL_RESULT iflymod_wifi_attach_msg(int id, OAL_CALL_FUNCTION pfunc)

void iflymod_wifi_enable_log(OAL_BOOL enable);
bool iflymod_wifi_is_connected();
void iflymod_wifi_connect(char *ssid, char *pass);

// print info api
void iflymod_wifi_show_apinfo();
void iflymod_wifi_show_currentstate();

```

应用程序在大多数情况是 attach WIFI 的状态汇报事件。目前的 jyd_sdk 中实现了微信配网和声波配网两种方式。

使用 JYD SDK 开发需注意事项：

- 无论是消息泵和注册回调，均支持，多对一，一对多，多对多，的事件的依附和注册，这样达到了软件模块的“各干各的事”的设计理念。例如有个事件“WIFI 连接成功”这个事件发生的时候，假如我们当前软件关心此事件或者此事件会对我“造成影响”，则依附该事件，可以很多模块很多地方都依附。不关心则不依附，省去模块内部调整，接口发生变化时任何调用处均需修改的状况。
- 嵌入式软件开发讲究运行效率和内存的节省，在处理各个环节，尽量保证精准判断，快速执行，快速返回，liboal 只对低速内存（psram）进行管理和使用，不对高速内存（sram）进行管理，高速内存供程序栈，变量栈，操作系统内核调度使用，没有特殊情况，不建议应用程序使用，对于内存的使用，请参考官方说，编译规则参考 appos.ld 的连接使用方法。

4 授权说明

iflyos_auth.h 中提供了授权接口，在整个 jyd_sdk 设计中存在两个授权：

- 金源德 sdk 的使用授权，授权 key 是一个串号。
- iFLYOS 的使用授权，包括 device_id 和 token。

整个授权过程就是将金源德 sdk 的串号和 iFLYOS 分配的 device_id 的串号写入设备 Flash 存储。

授权接口一：

到金源德公司服务器获取授权，授权过程中会发放 jyd_sdk 串号和 iFLYOS 的 device_id，并由设备自动获取 token（此接口设备量产时使用，开发者不必关心）。

接口：bool jyd_auth();

授权接口二：

通过已存储的 iFLYOS 的 device_id 产生 token。

接口：bool jyd_iflyos_generate_token()。

开发者手动授权的方法：

- jyd_sdk 的开发包的使用授权

参考示例工程中命令行“iflymodkey”的实现，格式为：

jyd iflymodkey <key>

key 为一个必填参数，往 Flash 里面写一个有效串号即可使用，该串号由金源德公司发布，默认情况下，金源德公司提供的硬件板等设备在出厂时已经写了该串号，开发者或者使用者不必关心，如果开发者使用的是自行设计的硬件板又希望使用金源德提供的 SDK，那么需要向金源德公司索要串号，然后用该命令写入，重启设备即可使用。

- iflyos 的登录授权

参考示例工程中命令行“iflyos”的实现，格式为：

jyd iflyos <device_id> <token>

其中 device_id 为必填参数，token 为可选参数，假如两个参数都存在，则直接写入 Flash 结束；假如只填一个参数 device_id，那么设备会联网自动获取一个 token 并写入 Flash。

授权成功后，重启设备即可使用。

5.3.4 运行 jyd_evm 示例工程

1. 编译工程

编译方法详见 3.4 节 SDK 编译

需注意：

目前 jyd_sdk 和样板程序均与 xr872 官方 SDK 处于同级目录

└jyd_evm	# jyd_evm 开发板应用软件工程目录
└jyd_sdk	# jyd_sdk 库和头文件目录
└xradio-skylark-sdk-xradio-skylark-sdk-1.0.2	# xr872 官方 SDK 的目录
└─readme.md	# 工程的说明文档

2. 烧录软件

烧录方法详见第 4 节固件烧录

3. 设备配网:

- a) 微信关注公众号：深圳市金源德实业有限公司；
- b) 开发板开机，长按配网键 4S 直至设备提示“进入微信配网”或“声波配网”模式；
- c) 按照配网引导进行配网操作；
- d) 设备端语音提示“配网成功”，配网完成；

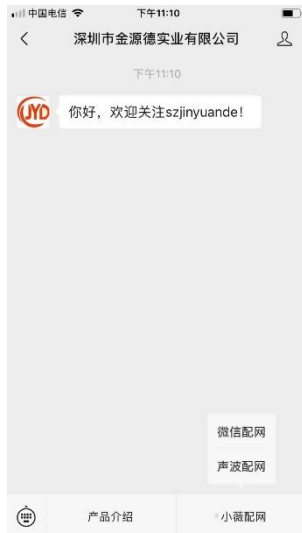


图 5-2



图 5-3



图 5-4

4. iFLYOS 语音交互体验

唤醒词：小微小微

唤醒后即可跟小微进行交互，你可以这样说：

天气查询：明天天气怎么样

听故事：讲个故事

更多技能体验可参考 iFLYOS 语音技能详情：<https://www.iflyos.cn/skills>

5.3.5 jyd_evm 二次开发

1. 修改 clientid

jyd_evm 项目下 jydsdk_common.c 文件里将 clientid 替换为自己的 clientid，替换位置如下：

```
jyd_sdk_common_data.clinet_id = "2f479800-b677-44af-b609-cee88708427a";
```

注意：client_id 需要向科大讯飞申请开通设备端隐式授权。

2. 替换 ota_secretkey

jyd_evm 项目下 jydsdk_common.c 文件里将 ota_secretkey 替换为自己的 ota_secretkey，替换位置如下：

```
jyd_sdk_common_data.ota_secretkey = "d6e0bdf20fdef993ea0fb8d403afc5a795d94f6a";
```

3. 将设备 deviceid 导入 iflyos 控制台

将需要接入 iflyos 的设备 deviceid 导入 iflyos 控制台，如下图所示：



图 5-5

4. 编译并烧录固件

将修改 clientid 和 ota_secretkey 后的 jyd_evm 项目编译并烧录到设备上。

5. 重新写入 token

步骤 1：查询设备 deviceid，并将 deviceid 发给讯飞生成 token。

命令：jyd deviceinfo

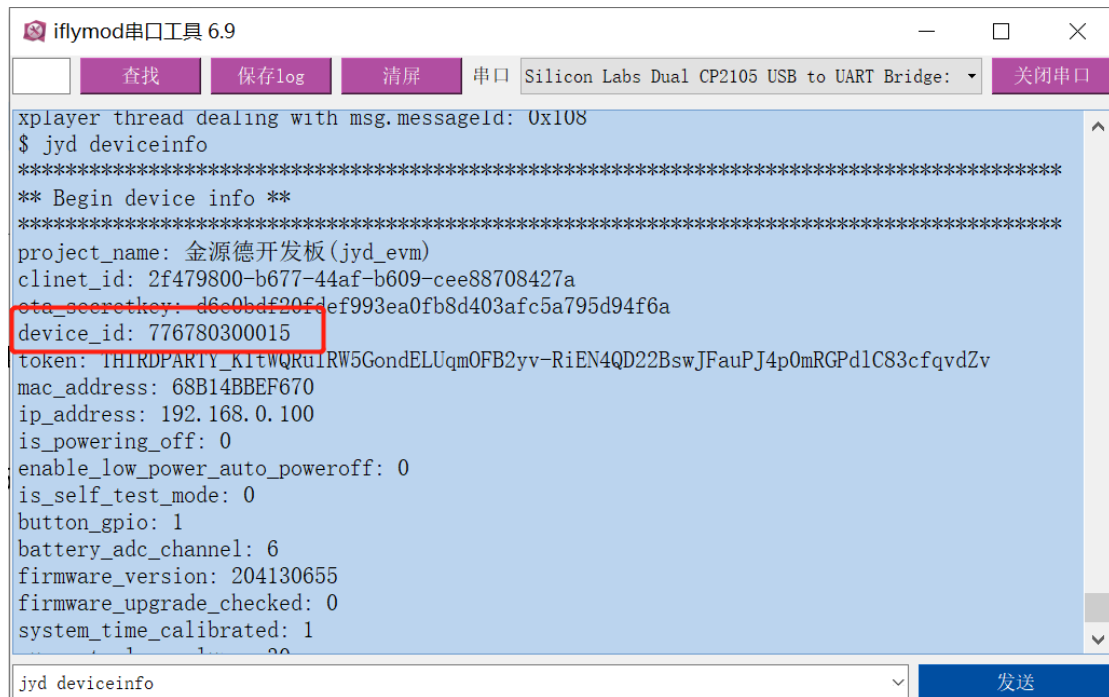


图 5-6

步骤 2：将 token 写入设备

命令: `jyd iflyos deviceid token`

示例:

```
jyd iflyos 776780300015 THIRDPARTY_***UCjtAp_X
```

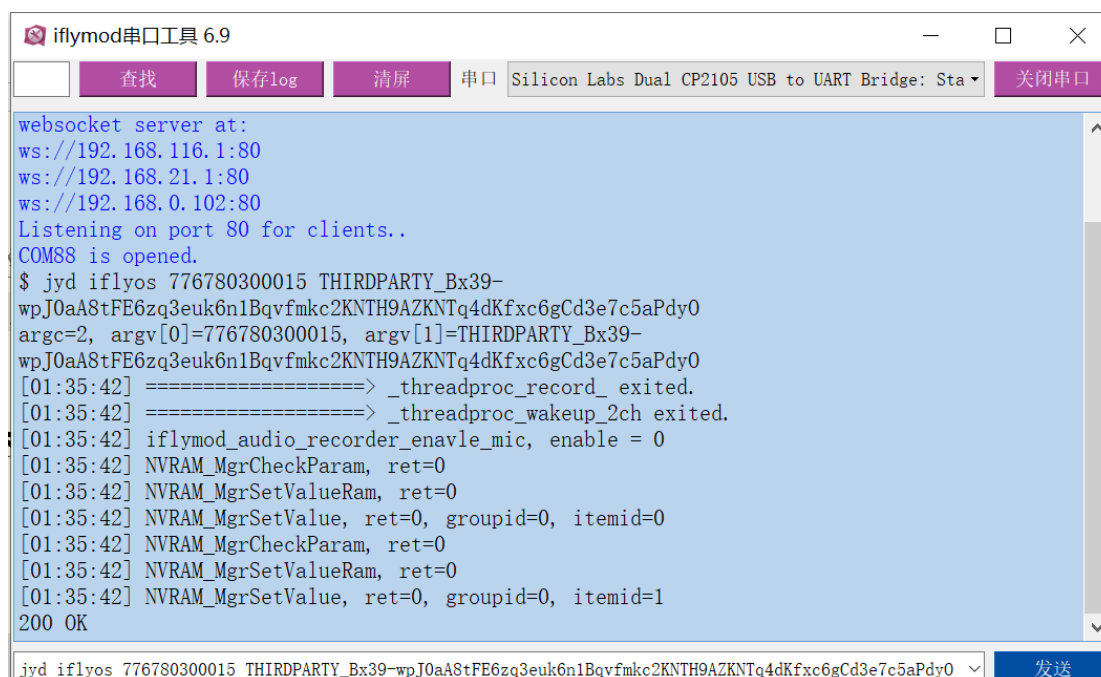


图 5-7

5.3.6 jyd_evm 控制命令

1. x872 官方命令

参考 xr872 SDK 中 project/example 中的示例代码

2. 金源德 jyd_evm 特有命令:

<code>\$ jyd connect <ssid> <password></code>	---	连接到指定 Wi-Fi
<code>\$ jyd deviceinfo</code>	---	显示设备信息
<code>\$ jyd clearallap</code>	---	删除已保存的所有 Wi-Fi 名称和密码
<code>\$ jyd iflyosota</code>	---	手动强制从讯飞后台进行固件更新
<code>\$ jyd voiceprint</code>	---	强制进入声波配网模式
<code>\$ jyd airkiss</code>	---	强制进入微信配网模式
<code>\$ jyd audio play <url></code>	---	播放一个 http 连接的音乐资源
<code>\$ jyd audio record</code>	---	从麦克风先录几秒钟的声音，再把录到的声音播放出来
<code>\$ jyd audio playrecord <url></code>	---	以 url 为背景音进行播放，同时从麦克风先录几秒钟的声音，再把录到的声音播放出来，并分三次播放：全部声音，麦克风声音，回采声音。麦克风声音在左声道，回采声音在右声道。
<code>\$ jyd audio volume <volume></code>	---	改变音量，0~100
<code>\$ jyd audio stop</code>	---	停止一切正在播放的内容
<code>\$ jyd audio i</code>	---	手动唤醒
<code>\$ jyd audio mic <0/1></code>	---	禁麦 / 使能麦


```
$ jyd audio mute <0/1>          --- 静音 / 使能音
$ jyd ota http <http://xxx.xxx/xr_system.img> --- OTA 到指定地址更新固件
```

5.3.7 提示音制作

参考 xr872 的官方镜像制作方法（在这里不详细阐述），我们规定提示音的 magic 为：0xa5f75a00（此命名大环境不变，不冲突的情况下永远不变）。

打包配置如下面这一行：{"id": "0xa5f75a00", "bin": "tone.bin", "cert": "null", "flash_offs": "1636K", "sram_offs": "0xffffffff", "ep": "0xffffffff", "attr": "0x0"}。

提示音随着系统固件一起打包，并留在系统镜像的最后部分，也能够随着 OTA 升级而变化。

提示音依据每个产品定位不同，需要采用一系列的 pcm 和 mp3 文件，并采用金源德提供的工具进行制作，产生 tone.bin 和 tone.h 替换源代码目录下 /image/xr872/tone.* 的对应两个文件。

工具 make_tone_packet 为命令行工具，位于 jyd_evm\image\xr872 目录下，后面加音频文件目录名称参数，即可产生打包文件。

其中目录中的内容如下：

名称 ^

-  00_咋了.pcm
-  01_干啥.pcm
-  02_对不起！播放出了点问题。 .mp3
-  03_进入微信配网模式.mp3
-  04_设备未能获得有效的云端授权.mp3
-  05_设备升级中，请保证电源连接，不要进行其它操作.mp3
-  06_网络不可用，请检查网络设置.mp3
-  07_网络连接成功.mp3
-  08_网络连接中，请稍等.mp3
-  09_网络出了点小状况，请再试试.mp3
-  10_你好，我是小微，是你的生活小帮手.mp3
-  11_授权码有误.mp3
-  12_进入声波配网模式.mp3
-  13_配网失败，请重新尝试.mp3
-  14_联网失败，请检查设置.mp3
-  15_小微时刻为你服务，再见.mp3
-  16_模块执行库未能获得有效授权.mp3
-  17_好的，设置完毕.mp3
-  18_密码错误，请检查设置.mp3
-  19_已收到网络名称，密码.mp3

使用命令：\$./make_tone_packet ****

示例：

```
Administrator@IT-201907231107 ~/jyd_evm/image/xr872
$ ./make_tone_packet tipvideo
```


其中 0 号和 1 号为 pcm，作为应答词，格式为 16k 采样率的单声道音频。其余提示音为 mp3 格式音频，序号不能变，文件内容和名称可以变。整理好这些文件，拿工具生成 tone.bin 打包到固件即可。

5.4 串口通信及说明

串口通信的代码在 “iflymod_extuartcomm.c” 和 “iflymod_extuartcomm.h” 文件中。串口通信默认采用 8 个数据位，1 个停止位，无校验，无流量控制。

常用接口说明如下：

打开串口：

```
void extcom_open(unsigned int baudRate, pfn_extcom_ProtocolInit pfn_ProtocolInit)
```

baudRate：串口通信波特率；

在串口初始化函数 pfn_ProtocolInit 里需要调用 extcom_setrxcallback 函数来设置串口接收数据后的回调。

关闭串口：

```
void extcom_close()
```

发送数据：

```
void extcom_sendbytes(unsigned char *pp_txbuf, unsigned int length)
```

设置接收到新数据回调函数：

```
void extcom_setrxcallback(pfn_extcom_RxCallback pfn_uartRxCallback)
```

代码示例：

打开串口：

```
void uartOpen(){
    extcom_open(115200, uart_extcom_init);
}

void uart_extcom_init()
{
    extcom_setrxcallback(uart_test_extcom_rxcallback);
}
```

发送指令：

```
/*自定义技能指令接收*/
static void cloud_callback(int id, int wParam, int lParam, void *data, int data_len)
{
    JYD_CLOUD_RESP_DATA *p = (JYD_CLOUD_RESP_DATA *)data;
    OAL_log("custom command %s\n",p->custom_string);
}
```

```

if(IFLYOS_RESPONSE_TYPE_CUSTOM == id){
    //open lights , close lights
    if(strcasecmp("openBedroomLight", p->custom_string)==0){
        OAL_log("打开卧室灯\n");
        extcom_sendbytes((unsigned char *)p->custom_string, strlen(p->custom_string));
    }else{
        OAL_log("自定义指令: %s\n", p->custom_string);
        extcom_sendbytes((unsigned char *)p->custom_string, strlen(p->custom_string));
    }
}
}
}

```

串口接收指令：

```

/*串口接收*/
static void uart_test_extcom_rxcallback(unsigned char *pp_rxbuf, unsigned int len)
{
    if(extcom_isopened() ==0)
        return;
    OAL_log("uart_test_extcom_rxcallback command= %s, strlen=%d \r\n", pp_rxbuf, strlen(pp_rxbuf));
    if(strncmp((const char *)pp_rxbuf, (const char *)extcommand_bedroom_close,
        strlen(extcommand_bedroom_close)) ==0)
    {
        OAL_log("关闭卧室灯\n");
        _play_promp_tone(36);
    }else if(strncmp((const char *)pp_rxbuf, (const char *)extcommand_aircondition_close,
        strlen(extcommand_aircondition_close)) ==0){
        OAL_log("关闭空调\n");
        _play_promp_tone(36);
    }
}
}

```

6 工程配置

6.1 本地配置选项

“config.mk”文件定义的所有全局配置选项，均可通过修改工程中的“project/[prj]/gcc/localconfig.mk”进行重定义，该定义将会覆盖“config.mk”中的默认值。例如，“config.mk”默认开启了 XIP 功能，定义如下：

```
__CONFIG_XIP ?= y
```

如果某工程要禁用 XIP 功能，则需在“project/[prj]/gcc/localconfig.mk”文件中增加以下定义：

```
export __CONFIG_XIP := n
```

注意：修改“project/[prj]/gcc/localconfig.mk”文件后，必须先执行“make build_clean”清除中间文件后，才可执行“make build”进行代码编译和镜像创建。

6.2 工程 Makefile

工程 Makefile 位于“project/[prj]/gcc/Makefile”，主要用于指定工程的板级配置、源文件、链接脚本、镜像配置文件等，关键变量的定义说明如表 6-1 所示。

变量名称	变量说明
ROOT_PATH	SDK 根目录路径，该路径是相对于工程 Makefile 的路径，例如“project/demo/hello_demo/gcc/Makefile”中定义的 ROOT_PATH 为“../../..”。
PROJECT	工程名字，默认定义为跟工程目录名字一致（一般无需修改），即： PROJECT := \$(notdir \$(shell cd .. && pwd))
PRJ_ROOT_PATH	工程根目录，使用变量“ROOT_PATH”进行定义。
PRJ_BOARD	（1）板级配置目录路径，该目录下必须包含“board_config.c”和“board_config.h”两个文件。 （2）由于不同方案的板级配置（主要是 pinmux 配置）一般都会存在差异，所以，新建的工程必须新建对应的板级配置，否则板级配置错误可能导致系统启动异常。 （3）在“project/common/board/”目录下存放了若干 XRADIO 开发板的板级配置文件，可参考这些文件完成特定工程板级配置文件的创建。
SRCS, OJS	定义工程源文件和目标文件。
PRJ_EXTRA_LIBS_PATH	定义额外的库文件搜索路径。默认库文件搜索路径为“lib/”。
PRJ_EXTRA_LIBS	（1）定义额外的库文件。 （2）如果工程需要使用 SDK 以外的库文件，可通过“PRJ_EXTRA_LIBS_PATH”和“PRJ_EXTRA_LIBS”变量来指定。例如，需要链接工程根目录下的“\$(PRJ_ROOT_PATH)/example/libbar.a”，则增加定义如下： “PRJ_EXTRA_LIBS_PATH := -L\$(PRJ_ROOT_PATH)/example” “PRJ_EXTRA_LIBS := -lbar”
PRJ_EXTRA_INC_PATH	定义额外的头文件搜索路径。例如，需要指定工程根目录下的“example”目录为头文件搜索路径，则增加定义如下： “PRJ_EXTRA_INC_PATH := -I\$(PRJ_ROOT_PATH)/example”
PRJ_EXTRA_SYMBOLS	定义额外的宏，仅工程代码可见，一般不使用。
LINKER_SCRIPT	（1）工程默认链接脚本为“project/linker_script/gcc/appos.ld”，特定工程可重定义“LINKER_SCRIPT”变量来指定该工程使用的链接脚本。 （2）“LINKER_SCRIPT”的路径定义是相对路径（相对于“\$(PRJ_ROOT_PATH)/gcc/”）。

	(3) 例如，工程需要使用“project/[prj]/gcc/appos.ld”链接脚本，则增加定义如下：LINKER_SCRIPT := ./appos.ld (4) 编译过程中，会根据默认链接脚本或者工程指定的链接脚本，生成最终使用的链接脚本“project/[prj]/gcc/.project.ld”。 (5) 链接产生的 ELF 文件、bin 文件、map 文件等位于“project/[prj]/gcc/”目录下。
IMAGE_CFG	(1) 工程默认镜像配置文件为“project/image_cfg/image.cfg”，特定工程可重定义“IMAGE_CFG”变量来指定该工程使用的镜像配置文件。 (2) “IMAGE_CFG”的路径定义是相对路径（相对“\$(PRJ_ROOT_PATH)/image/[chip]/”）。 例如，工程需要使“project/[prj]/image/xr872/image.cfg”镜像配置文件，则增加定义如下：IMAGE_CFG := ./image.cfg 镜像创建过程中，会根据默认镜像配置文件或者工程指定的镜像配置文件，生成最终使用的镜像配置文件“project/[prj]/image/[chip]/.image.cfg”。 创建的镜像文件位于“project/[prj]/image/[chip]/”目录下。
IMAGE_NAME	(1) 工程默认镜像名字为“xr_system.img”，特定工程可重定义“IMAGE_NAME”变量来指定镜像名字（不含扩展名，扩展名固定为“img”）。 (2) 例如，工程需要指定镜像名字为“examplebar.img”，则增加定义如下：IMAGE_NAME := examplebar

表 6-1 工程 Makefile 变量

6.3 链接脚本

链接脚本（扩展名为“.ld”）主要用于描述如何将输入文件（目标文件、库文件）中的 section 映射到输出文件（ELF 文件）中，并控制输出文件的存储布局。

工程的默认链接脚本为：“project/linker_script/gcc/appos.ld”；特定工程可以通过定义“project/[prj]/gcc/Makefile”中的“LINKER_SCRIPT”变量来指定该工程使用的链接脚本，表 6-1 中有关于“LINKER_SCRIPT”变量的详细说明。

6.4 镜像配置文件

镜像配置文件（扩展名为“.cfg”）采用 JSON 格式书写，主要用于描述如何将工程编译生成的二进制文件（app.bin、app_xip.bin 等）和预置的二进制文件（wlan_bl.bin、wlan_fw.bin 等）打包成镜像文件（扩展名为“.img”）。

工程默认镜像配置文件为：“project/image_cfg/image.cfg”；特定工程可通过重定义“project/[prj]/gcc/Makefile”中的“IMAGE_CFG”变量来指定该工程使用的镜像配置文件。表 6-1 中有关于“IMAGE_CFG”变量的详细说明。

镜像配置的详细说明文档可参考文档：

《XRADIO_Memory_Layout_Developer_Guide-CN》

《XRADIO_Flash_Layout_Guide-CN》，镜像文件由“tools/mkimage”工具生成，通

过烧录工具“tools/phoenixMC.exe”可将镜像文件烧录到开发板的 FLASH 中运行。

6.5 创建新工程

6.5.1 创建工程目录

新工程的创建可参考 project 目录下的已有工程。首先需要给工程起个有意义的名字，并以该名字作为工程目录。工程名字建议和项目相关，例如 storyteller（故事机工程）；不推荐采用不具备标识性的名字，例如 demo1、audio2 等。假设需要创建新工程 example，操作如下：

```
# 以 hello_dem 为模板，创建新工程 example
$ cp -r project/demo/hello_demo project/example
```

6.5.2 创建工程板级配置文件

由于不同项目方案的 PCB 板设计都不一样，一般情况下，每个项目方案都需要创建自己的板级配置文件。板级配置文件的创建可参考“project/common/board/”目录下的 XRADIO 开发板配置文件。假设需要在新工程 example 创建板级配置 bar_evb，操作如下：

```
#以 project/common/board/xradio_evb 为模板，创建板级配置 project/example/bar_evb
# （假设当前工作目录为 SDK 根目录）
$ cp -r project/common/board/xradio_evb project/example/bar_evb
```

执行以上操作后，创建“project/example/bar_evb”目录，目录下包含“board_config.c”和 board_config.h 两个文件，所有的板级配置目录都必须包含这两个文件。

由于从“project/common/board/xradio_evb”复制的板级配置文件只适用于 XRADIO 开发板，所以，必须根据实际的 PCB 设计修改板级配置文件。

“board_config.c”和“board_config.h”共同定义了板级相关的配置信息，例如 pinmux 配置、CPU 频率、AHB/APB 总线频率、UART 波特率等。最关键的是要修改“board_config.c”中的“pinmux”配置，使之与 PCB 设计相符。

```
static const GPIO_PinMuxParam g_pinmux_uart0[] = {
    { GPIO_PORT_B, GPIO_PIN_0, { GPIOB_P0_F2_UART0_TX, GPIO_DRIVING_LEVEL_1, GPIO_PULL_UP } }, /* TX */
    { GPIO_PORT_B, GPIO_PIN_1, { GPIOB_P1_F2_UART0_RX, GPIO_DRIVING_LEVEL_1, GPIO_PULL_UP } }, /* RX */
};

static const GPIO_PinMuxParam g_pinmux_uart1[] = {
    { GPIO_PORT_A, GPIO_PIN_7, { GPIOA_P7_F2_UART1_TX, GPIO_DRIVING_LEVEL_1, GPIO_PULL_UP } }, /* TX */
    { GPIO_PORT_A, GPIO_PIN_6, { GPIOA_P6_F2_UART1_RX, GPIO_DRIVING_LEVEL_1, GPIO_PULL_UP } }, /* RX */
    { GPIO_PORT_A, GPIO_PIN_5, { GPIOA_P5_F2_UART1_CTS, GPIO_DRIVING_LEVEL_1, GPIO_PULL_DOWN } }, /* CTS */
    { GPIO_PORT_A, GPIO_PIN_4, { GPIOA_P4_F2_UART1_RTS, GPIO_DRIVING_LEVEL_1, GPIO_PULL_DOWN } }, /* RTS */
};

static const GPIO_PinMuxParam g_pinmux_uart2[] = {
    { GPIO_PORT_A, GPIO_PIN_22, { GPIOA_P22_F2_UART2_TX, GPIO_DRIVING_LEVEL_1, GPIO_PULL_UP } }, /* TX */
    { GPIO_PORT_A, GPIO_PIN_21, { GPIOA_P21_F2_UART2_RX, GPIO_DRIVING_LEVEL_1, GPIO_PULL_UP } }, /* RX */
    { GPIO_PORT_A, GPIO_PIN_20, { GPIOA_P20_F2_UART2_CTS, GPIO_DRIVING_LEVEL_1, GPIO_PULL_DOWN } }, /* CTS */
    { GPIO_PORT_A, GPIO_PIN_19, { GPIOA_P19_F2_UART2_RTS, GPIO_DRIVING_LEVEL_1, GPIO_PULL_DOWN } }, /* RTS */
};
```

```
static HAL_Status board_get_pinmux_info(uint32_t major, uint32_t minor, uint32_t param,
                                       struct board_pinmux_info info[])
{
    HAL_Status ret = HAL_OK;

    switch (major) {
    case HAL_DEV_MAJOR_UART:
        if (minor == UART0_ID) {
            info[0].pinmux = g_pinmux_uart0;
            info[0].count = HAL_ARRAY_SIZE(g_pinmux_uart0);
        } else if (minor == UART1_ID) {
            info[0].pinmux = g_pinmux_uart1;
            info[0].count = HAL_ARRAY_SIZE(g_pinmux_uart1);
        } else if (minor == UART2_ID) {
            info[0].pinmux = g_pinmux_uart2;
            info[0].count = HAL_ARRAY_SIZE(g_pinmux_uart2);
        } else {
            ret = HAL_INVALID;
        }
        break;
    }
}
```

图 6-1 工程 pinmux 配置示例

图 6-1 展示了“project/common/board/xradio_evb/board_config.c”中 uart 相关的 pinmux 配置，主要包含两部分代码的实现：

1. pinmux 配置数组，图中包含了 uart0、uart1、uart2 三个 pinmux 配置组。
2. “board_get_pinmux_info()”函数中对 pinmux 配置数组的获取代码。所有的 pinmux 配置信息均需包含以上两部分的代码实现。

修改工程“board_config.c”的注意事项包括：

1. 所有的 pinmux 配置信息不能冲突，任何一个 pin 脚只能配置一种功能。例如：在 uart0 pinmux 配置数组中将 PB01 设置为“PIOB_P1_F2_UART0_RX”，就不能同时在 PWM pinmux 配置数组中将 PB01 设置为“GPIOB_P1_F4_PWM5_ECT5”。
2. 板级配置信息（pinmux 配置等）必须与 PCB 设计相符，否则会导致系统启动异常或功能不可用。
3. 删除无用的 pinmux 配置信息以节省代码空间(为了示范不同外设的 pinmux 配置方法，“project/common/board/xradio_evb/board_config.c”定义了较全的外设 pinmux 配置，实际使用不会这么多)。例如：PCB 只设计一个 UART0，则 UART1、UART2 相关的配置信息都应删除。
4. 创建或者删除 pinmux 配置数组的同时，务必修改“board_get_pinmux_info()”函数的相关代码，否则会导致编译错误。

6.5.3 创建工程链接脚本

工程默认链接脚本为“project/linker_script/gcc/appos.ld”。新建工程一般需要实现本工程的链接脚本，相关说明可参考 5.2.2 节“LINKER_SCRIPT”变量说明和 5.2.3 节。

建议新工程创建本工程的链接脚本路径为：“project/[prj]/gcc/appos.ld”，具体实现可参考以下两个文件：

1. 工程默认链接脚本：“project/linker_script/gcc/appos.ld”
2. 其他工程编译过程中产生的实际链接脚本：“project/[prj]/gcc/.project.ld”

6.5.4 创建工程镜像配置文件

工程默认镜像配置文件为“project/image_cfg/image.cfg”。新建工程一般要实现本工程的镜像配置文件，相关说明可参考 6.2 节“IMAGE_CFG”变量说明和 6.3 节。建议新工程创建本工程的镜像配置文件路径为：

“project/[prj]/image/[chip]/image.cfg”，具体实现可参考以下两个文件：

1. 工程默认镜像配置文件：“project/image_cfg/image.cfg”；
2. 其他工程创建镜像过程中产生的实际镜像配置文件：

“project/[prj]/image/[chip]/.image.cfg”。

6.5.5 修改工程 localconfig.mk

工程本地配置选项在“project/[prj]/gcc/localconfig.mk”中定义，可覆盖“config.mk”中的默认值。具体工程需要根据项目方案需求，开启或禁用相关配置选项。例如，以下配置开启了 xplayer、XIP、OTA 功能。

```
# enable/disable xplayer, default to n
export __CONFIG_XPLAYER := y

# enable/disable XIP, default to y
export __CONFIG_XIP := y

# enable/disable OTA, default to n
export __CONFIG_OTA := y
```

6.5.6 修改工程 Makefile

工程 Makefile 位于“project/[prj]/gcc/Makefile”，主要用于指定工程的板级配置、源文件、链接脚本、镜像配置文件等，详细说明可参考 6.2 节。

一般需要修改的项包括：ROOT_PATH、PRJ_ROOT_PATH、PRJ_BOARD、SRCS、LINKER_SCRIPT、IMAGE_CFG。假设“project/example”工程的文件目录结构（假设芯片选择为 XR872）如下：

```
project/example/
├── bar_evb
│   ├── board_config.c
│   └── board_config.h
├── command.c
├── command.h
├── gcc
│   ├── appos.ld
│   ├── localconfig.mk
│   └── Makefile
└── image
```

```

|   └─ xr872
|   └─ image.cfg
|   └─ main.c
└─ prj_config.h

```

“project/example/gcc/Makefile”中需要修改的变量如下：

```

# 定义 SDK 根目录
ROOT_PATH := ../../..

# 定义工程根目录
PRJ_ROOT_PATH := $(ROOT_PATH)/project/$(PROJECT)

# 定义板级配置文件目录
PRJ_BOARD := $(PRJ_ROOT_PATH)/bar_evb

# 定义工程源文件，需要根据具体工程的源代码结构进行源代码指定
SRCS := $(basename $(foreach dir,$(DIRS),$(wildcard $(dir)/*.c)))

# 定义链接脚本
LINKER_SCRIPT := ./appos.ld

# 定义镜像配置文件
IMAGE_CFG := ./image.cfg

```

6.5.7 修改工程 prj_config.h

工程“prj_config.h”定义本工程的功能选项配置值，可覆盖“project/common/prj_conf_opt.h”中的默认值。具体工程需要根据项目方案需求，重定义相关的功能选项。

所有可选的功能选项可参考“project/common/prj_conf_opt.h”中的注释描述。例如，以下代码将主线程栈设置为 2KB，MAC 地址从 EFUSE 获取，禁用看门狗超时复位功能：

```

/* main thread stack size */
#define PRJCONF_MAIN_THREAD_STACK_SIZE (2 * 1024)

/* MAC address source */
#define PRJCONF_MAC_ADDR_SOURCE SYSINFO_MAC_ADDR_EFUSE

/* watchdog enable/disable */
#define PRJCONF_WDG_EN 0

```


6.5.8 修改工程 command.c

工程“command.c”定义了本工程可用的控制台命令。具体工程可根据需求增加或删除命令，SDK 支持的所有命令在“project/common/cmd/”目录下实现，用户也可以自定义新命令。

7 常见问题及解决方法

7.1 如何替换 clientid

步骤 1: 代码中替换 clientid 和 ota_secretkey

首先[创建 clientid](#)，其次在 jyd_evm 项目下 jydsdk_common.c 文件里将 clientid 替换为自己的 clientid，替换位置如下：

```
jyd_sdk_common_data.clinet_id = "2f479800-b677-44af-b609-cee88708427a";
```

注意：client_id 需要向科大讯飞申请开通设备端隐式授权。

步骤 2: 替换 ota_secretkey

jyd_evm 项目下 jydsdk_common.c 文件里将 ota_secretkey 替换为自己的 ota_secretkey，替换位置如下：

```
jyd_sdk_common_data.ota_secretkey = "d6e0bdf20fdef993ea0fb8d403afc5a795d94f6a";
```

步骤 3: 将设备 deviceid 导入 iflyos 控制台

将需要接入 iflyos 的设备 deviceid 导入 iflyos 控制台，如下图所示：

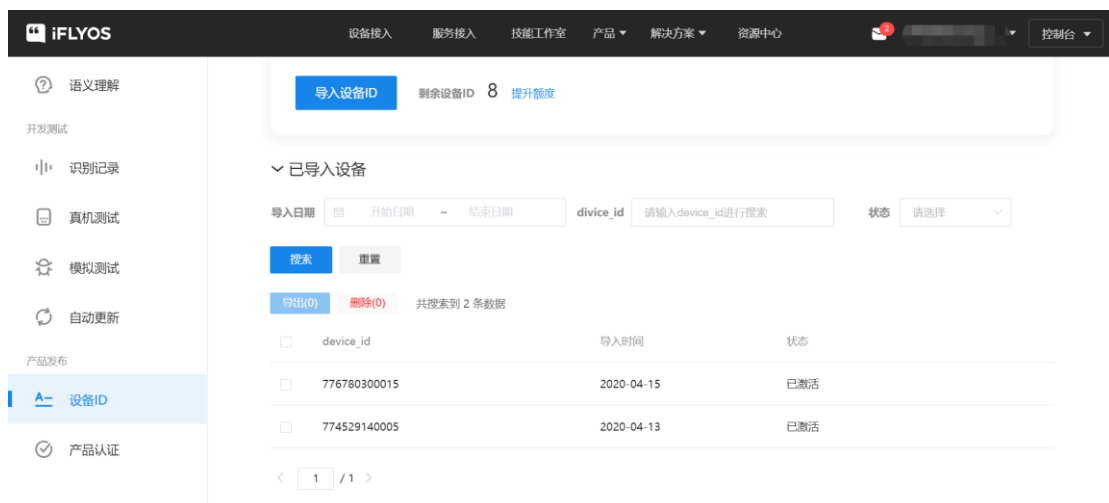


图 7-1

步骤 4：编译并烧录固件

将修改 clientid 和 ota_secretkey 后的 jyd_evm 项目编译并烧录到设备上。

烧录命令：

```
jyd ota http http://192.168.1.133:8000/upload/xr_system.img
```

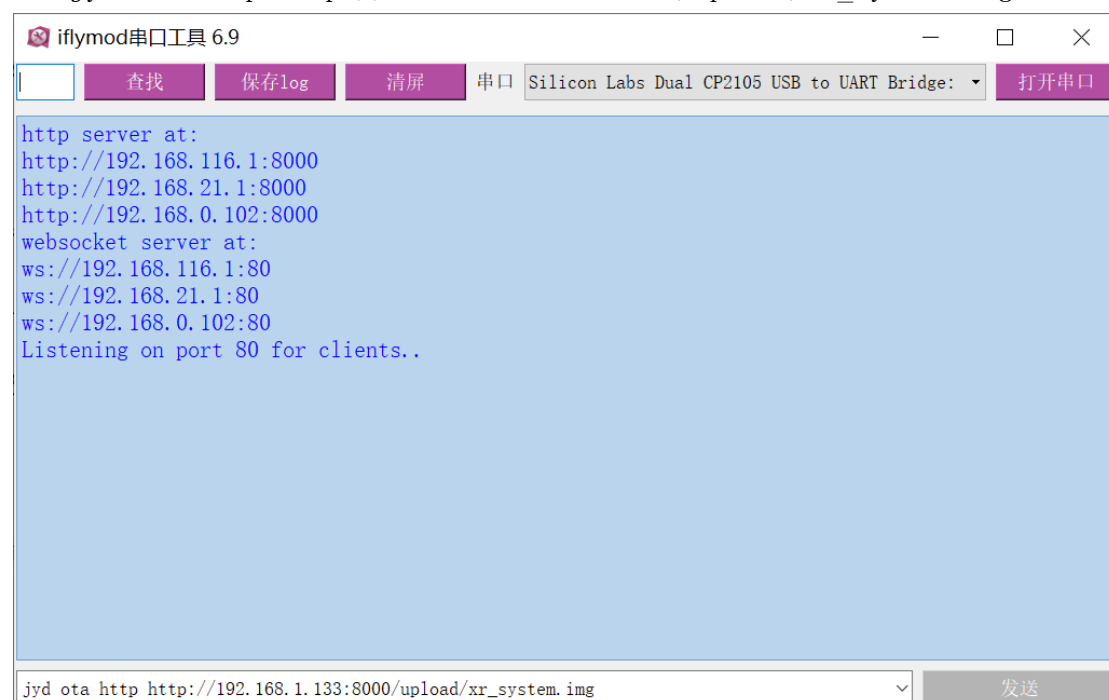


图 7-2

命令说明：

4. http://192.168.1.133 电脑 ip 地址，端口默认未 8000。
5. upload 目录为 iflymod 串口工具安装目录下用了存放固件的文件夹，若无则创建，如：D:\Roaming\iUartTool。

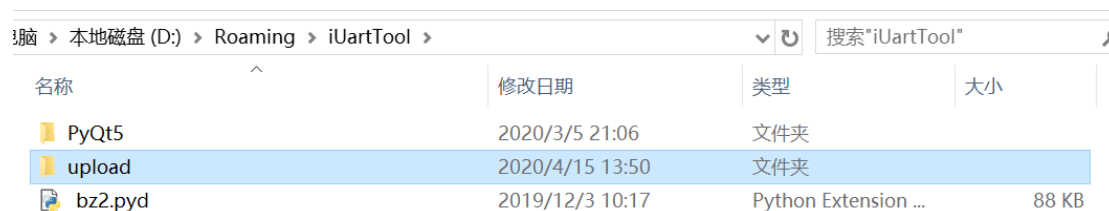


图 7-3

6. xr_system.img 为需要烧录固件名称。

步骤 5：查询设备 ID 并申请 token

查询设备 deviceid，并将 deviceid、clientid 发给讯飞生成 token。

命令：jyd deviceinfo

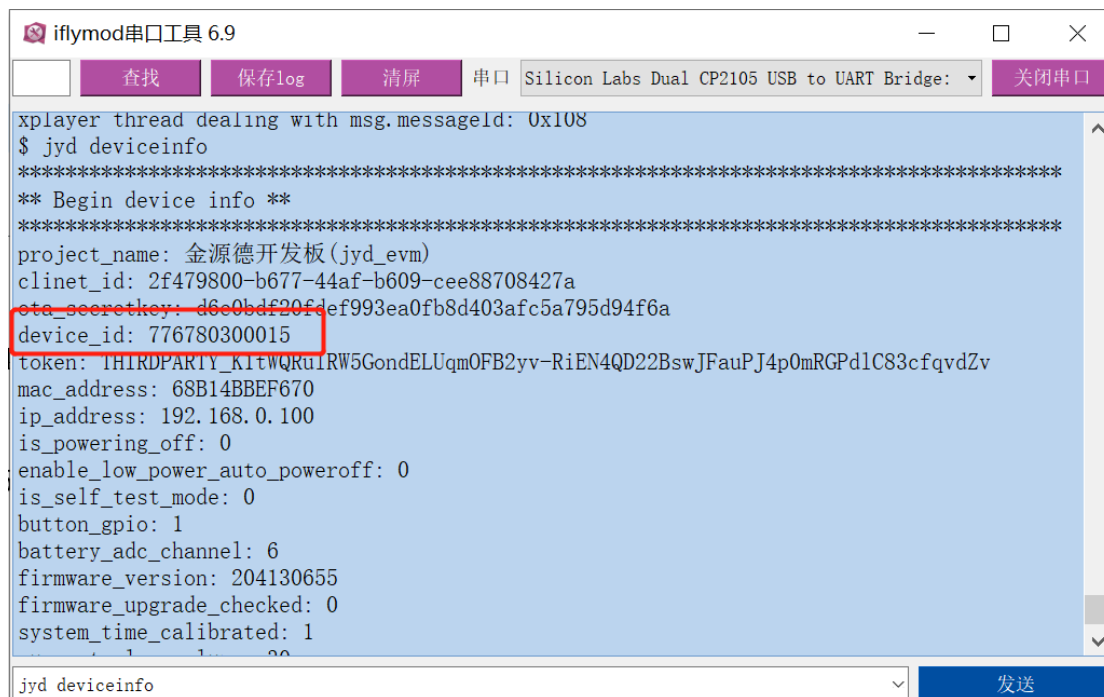


图 7-4

步骤 6: 将 token 写入设备

命令: `jyd iflyos deviceid token`

示例:

```
jyd iflyos 776780300015 THIRDPARTY_***UCjtAp_X
```

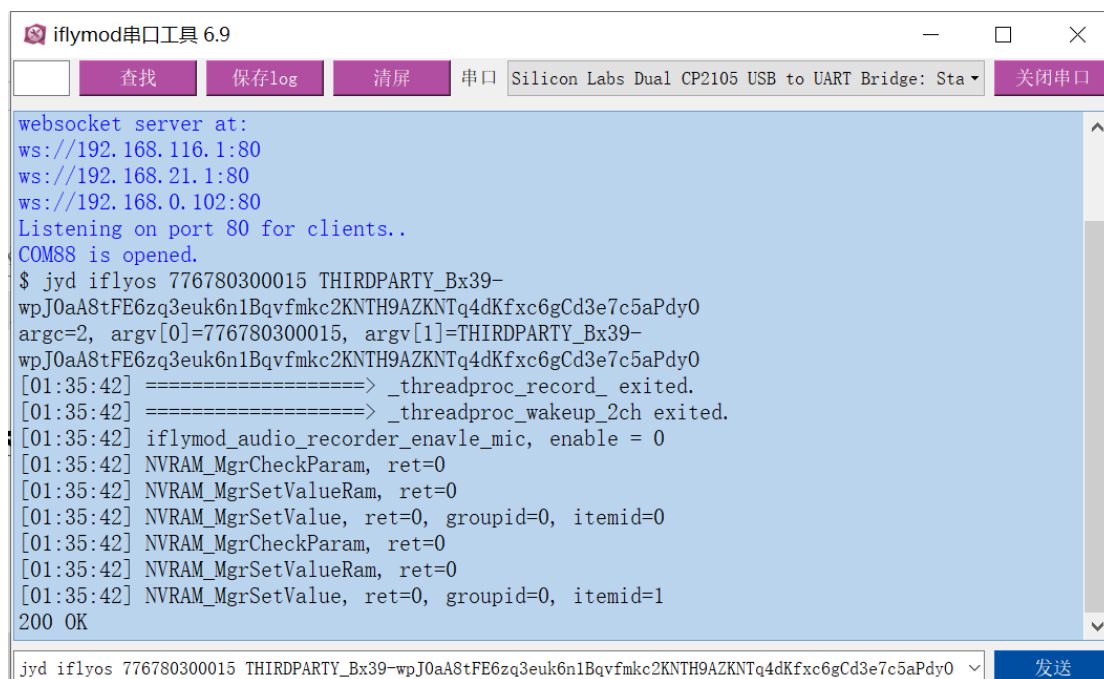


图 7-5

Token 写入成功后即可正常交互。

7.2 如何自定义唤醒词和离线命令词

自定义唤醒词和离线命令词需要向讯飞对应技术支持工程师提出申请并提供自定义内容，由讯飞输出自定义唤醒词包，拿到唤醒词包后替换对应工程 libwakeup 目录下的唤醒库文件，重新编译项目并烧录到设备上即可生效。唤醒词库文件包含以下内容：

- keyword.txt
- keyword_to_pinyin.txt
- libaec.a
- libivw.a
- libwake_util.a
- version.txt
- wakeup_util.h

图 7-6

7.3 如何开发自定义技能

前往技能工作室编写自定义技能及后处理，并将写好的技能配在设备接入平台的拦截器上。具体流程可参考[技能开发指引](#)。

示例：智能家居控制自定义技能实现
智能家居控制技能

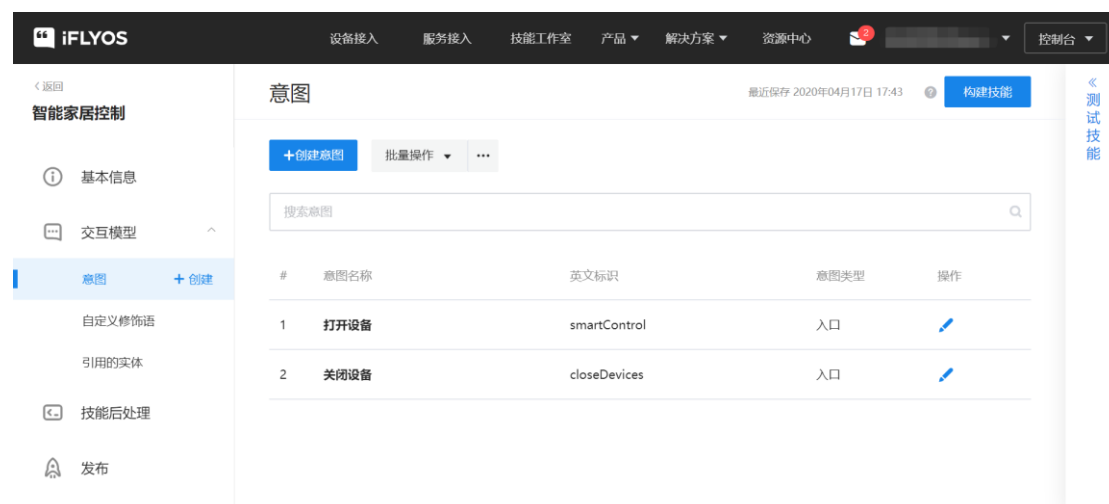


图 7-7

打开设备意图



图 7-8

智能家居设备实体

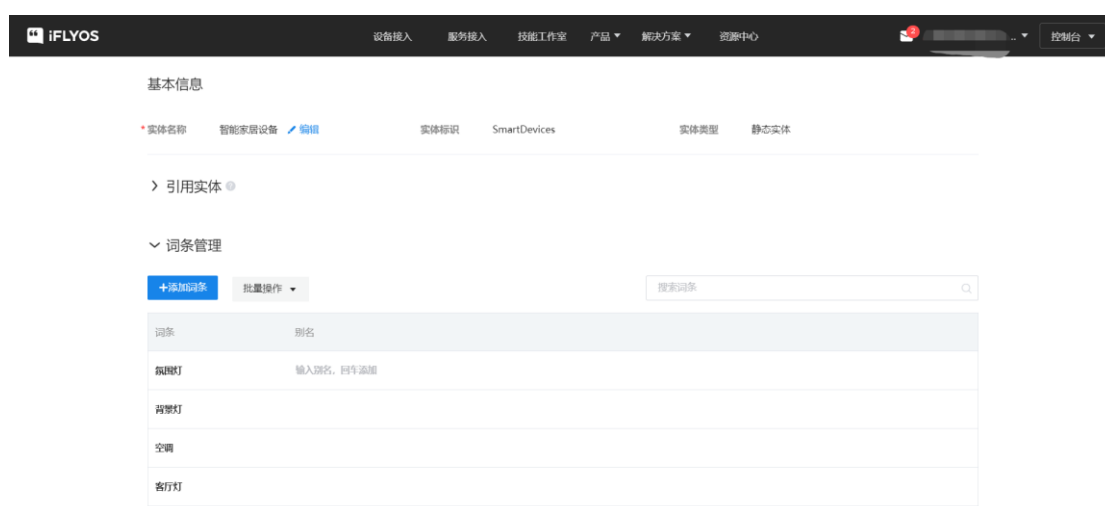


图 7-9

技能后处理实现代码：

可根据自定义指令控制需求修改技能后处理代码，实现自定义控制指令下发。

```
AIUI.create("v2.1", function(aiui, err){
    requestObject = aiui.getRequest().getObject();
    var response = aiui.getResponse();
    // 判断请求类型
    var requestType = requestObject.request.type;
    console.log("技能请求类型为:" + requestType);
    if(requestType === "LaunchRequest"){
        // 会话保持活动状态
        response.withShouldEndSession(false);
    } else if(requestType === "IntentRequest"){
        // 会话保持活动状态
        response.withShouldEndSession(false);
        // 获取当前意图名
```

```
intentName = requestObject.request.intent.name;
original = requestObject.request.query.original;
console.log("本次意图来自:" + intentName);
console.log("语义:" + original);

//开灯
if(intentName === "smartControl"){
    var slotName_execution = requestObject.request.intent.slots.Execution.value
    var slotName_devices = requestObject.request.intent.slots.devices.value
    console.log("slotName " + slotName)

    if(slotName_execution === "打开"){
        if(slotName_devices === "台灯"){
            response.setDirectives([
                {
                    "type": "Custom",
                    "header": {
                        "namespace": "Custom",
                        "name": "Custom",
                    },
                    "payload": {
                        "intent": "openDeskLight" // 自定义数据
                    }
                }
            ]);
        }else if(slotName_devices === "卧室灯"){
            response.setDirectives([
                {
                    "type": "Custom",
                    "header": {
                        "namespace": "Custom",
                        "name": "Custom",
                    },
                    "payload": {
                        "intent": "openBedroomLight" // 自定义数据
                    }
                }
            ]);
        }else if(slotName_devices === "空调"){
            response.setDirectives([
                {
                    "type": "Custom",
                    "header": {
                        "namespace": "Custom",
                        "name": "Custom",
                    },
                    "payload": {
                        "intent": "openAircondition" // 自定义数据
                    }
                }
            ]);
        }
    }
}
```

```
    });  
    }  
  }  
  response.setOutputSpeech("正在为您" + slotName_execution + slotName_devices);  
}  
  
//关灯  
if(intentName === "closeDevices"){  
  var slotName = requestObject.request.intent.slots.devices.value  
  console.log("slotName " + slotName)  
  
  if(slotName === "台灯"){  
    response.setDirectives([ {  
      "type": "Custom",  
      "header": {  
        "namespace": "Custom",  
        "name": "Custom",  
      },  
      "payload": {  
        "intent": "closeDeskLight" // 自定义数据  
      }  
    }  
  ]  
  );  
  } else if(slotName === "卧室灯"){  
    response.setDirectives([ {  
      "type": "Custom",  
      "header": {  
        "namespace": "Custom",  
        "name": "Custom",  
      },  
      "payload": {  
        "intent": "closeBedroomLight" // 自定义数据  
      }  
    }  
  ]  
  );  
  } else if(slotName === "空调"){  
    response.setDirectives([ {  
      "type": "Custom",  
      "header": {  
        "namespace": "Custom",  
        "name": "Custom",  
      },  
      "payload": {  
        "intent": "closeAircondition" // 自定义数据  
      }  
    }  
  ]  
  );  
}
```

```

    }
    response.setOutputSpeech("正在为您关闭" + slotName);
}

} else if(requestType === "SessionEndedRequest"){
    response.withShouldEndSession(true);
}
aiui.commit();
})

```

产品关联自定义技能



图 7-10

7.4 jyd_evm 示例项目在哪里接收自定义技能控制指令

自定义技能控制指令接收代码位置 main_task.c 中的 cloud_callback 方法，可根据自己的需求接收指令并做相应处理，示例 jyd_evm 示例中将自定义技能指令通过串口通信发送到外部设备实现控制操作：

```

static void cloud_callback(int id, int wParam, int lParam, void *data, int data_len)
{
    JYD_CLOUD_RESP_DATA *p = (JYD_CLOUD_RESP_DATA *)data;
    OAL_log("custom command %s\n", p->custom_string);
    if(IFLYOS_RESPONSE_TYPE_CUSTOM == id){
        //open lights , close lights
        if(strcasecmp("openBedroomLight", p->custom_string)==0){
            OAL_log("打开卧室灯\n");
            extcom_sendbytes((unsigned char *)p->custom_string, strlen(p->custom_string));
        }
        else if(strcasecmp("openDeskLight", p->custom_string)==0){
            OAL_log("打开台灯\n");
        }
    }
}

```



```

        extcom_sendbytes((unsigned char *)p->custom_string, strlen(p->custom_string));
    }else if(strcasecmp("closeBedroomLight", p->custom_string)==0){
        OAL_log("关闭卧室灯\n");
        extcom_sendbytes((unsigned char *)p->custom_string, strlen(p->custom_string));
    }else if(strcasecmp("closeDeskLight", p->custom_string)==0){
        OAL_log("关闭台灯\n");
        extcom_sendbytes((unsigned char *)p->custom_string, strlen(p->custom_string));
    }else{
        OAL_log("其他自定义指令: %s\n",p->custom_string);
        extcom_sendbytes((unsigned char *)p->custom_string, strlen(p->custom_string));
    }
}
}
}

```

7.5 jyd_evm 示例项目如何处理离线命令

离线指令处理代码位置 main_task.c 中的 recorder_callback 方法，wParam 为指令编号，指令编号可在唤醒词资源文件中的 keyword.txt 查看，开发者可根据自己的需求接收指令并做相应处理。

```

/*离线命令处理*/
static void recorder_callback(int id, int wParam, int lParam, void *data, int data_len)
{
    if(MSG_AUDIO_RECOGNIZE_VOICE_SEND_FAILED == id){
        iflymod_playlist_add_promp_tone(9);
    }else if(MSG_AUDIO_OFFLINE_COMMAND == id){
        // add offline command doing here...
        //离线命令处理
        if(wParam == 2)
        {
            OAL_log("offline command:打开台灯\r\n");
            _play_promp_tone(21);
        }else if(wParam == 3)
        {
            OAL_log("offline command:灯光明亮\r\n");
            _play_promp_tone(22);
        }else if(wParam == 4)
        {
            OAL_log("offline command:灯光柔和\r\n");
            _play_promp_tone(23);
        }else if(wParam == 10){

```

```
OAL_log("offline command:温度低一点\r\n");
_play_promp_tone(26);
    }
}
}
```